Words of Concern

Point to be As Allan Bloom has said "Education is the movement from darkness to light". Through this handbook, I have tried to illuminate what might otherwise appear as black boxes to some. In doing so, I have used references from several other authors to synthesize or simplify or elaborate information. This is not possible without omitting details that I deem trivial while dilating the data that I consider relevant to topic. Every effort has been made to avoid errors. In spite of this, some errors might have crept in. Any errors or discrepancies noted maybe brought to my notice which I shall rectify in my next revision.

This handbook is solely for educational purpose and is not for sale. This handbook shall not be reproduced or distributed or used for commercial purposes in any form or by any means.

Thanks,
Raghu Gurumurthy
Interface College of Computer Applications (ICCA)
Davangere



Bibliography

Ian Somerville, —Software Engineering‖ 8th Edition, Pearson Education, 2009.

References Books

Waman S Jawadekar, —Software Engineering Principles and Practice‖, Tata McGrawHill, 2004.
Roger S. Pressman, —A Practitioners Approach‖,7th Edition, McGraw-Hill, 2007.
P Jalote, —An Integrated Approach to software Engineering‖, Narosa Publication.

Interface College of Computer Applications (ICCA)

Syllabus:

Course Title : Software Engineering (Theory)
Course Code: DSC15
No. of Credits: 04
Contact hours: 52 Hours(4hrs/Week)
Duration of SEA/Exam: 2 hours
Formative Assessment Marks: 40
Summative Assessment Marks: 60
Credits: 4

Course Outcomes (COs): After the successful completion of the course, the student will be able to:

UNIT-1
OVERVIEW: Introduction; Software engineering ethics; Software process models; Process activities; Coping with change; Agile software development: Agile methods; Plan-driven and agile development. 10 Hours

UNIT-2
REQUIREMENTS ENGINEERING: Functional and non-functional requirements; Software requirements document; Requirement's specification; Requirements engineering processes; Requirement's elicitation and analysis; Requirement's validation; Requirements management. 10 Hours

UNIT-3
SYSTEM MODELING: Context models; Interaction models- Use case modeling, Sequence diagrams; Structural models- Class diagrams, Generalization, Aggregation; Behavioral models- Data-driven modeling, Event-driven modeling; Model-driven engineering. 10 Hours

UNIT-4
ARCHITECTURAL DESIGN: Architectural design decisions; Architectural views; Architectural patterns- Layered architecture, Repository architecture, Client–server architecture Pipe and filter architecture.

DESIGN AND IMPLEMENTATION: Object-oriented design using the UML- System context and interactions, Architectural design, Object class identification, Design models, Interface specification; Design patterns; Implementation issues. 12 Hours

UNIT-5
SOFTWARE TESTING: Development testing- Unit testing, Choosing unit test cases, Component testing, System testing. Test-driven development; Release testing; User testing- Alpha, Beta, Acceptance testing.

ICCA
Interface College of Computer Applications (ICCA)

Unit - I: OVERVIEW

Introduction

1. What is software?

Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

2. What are the attributes of good software?

Good software should deliver the required functionality and performance to the user and should be **maintainable**, **dependable**, and **usable**.

3. What is software engineering?

Software engineering is an **engineering discipline** that is concerned with all aspects of **software production**.

4. What are the fundamental software engineering activities?

Software **specification**, software **development**, software **validation**, and software **evolution**.

5. What is the difference between software engineering and computer science?

Computer science focuses on theory and fundamentals; software engineering is concerned with the **practicalities** of developing and delivering useful software.

6. What is the difference between software engineering and system engineering?

System engineering is concerned with all aspects of computer-based systems development including hardware, software, and process engineering. Software engineering is part of this more general process.

7. What are the key challenges facing software engineering?

Interface College of Computer Applications (ICCA)

Coping with increasing diversity, demands for reduced delivery times, and developing trustworthy software.

8. What are the costs of software engineering?

Roughly 60% of software costs are **development** costs; 40% are **testing** costs. For custom software, evolution costs often exceed development costs.

9. What are the best software engineering techniques and methods?

While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analysable specification to be developed. We can't, therefore, say that one method is better than another.

10. What differences has the Web made to software engineering?

The Web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Types of Software Products

There are two kinds of software products.

Generic products

- These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them. Examples of this type of product include software for PCs such as databases, word processors, drawing packages, and project-management tools.

- It also includes so-called vertical applications designed for some specific purpose. such as library information systems, accounting systems, or systems for maintaining dental records.

Customized (or bespoke) products

- These are systems that are **commissioned** by a particular customer. A software contractor develops the software especially for that customer. Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

Essential attributes of good software

- Product characteristics Description
  Maintainability Software should be written in such a way so that it can evolve to **meet the changing needs** of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

Interface College of Computer Applications (ICCA)

- Dependability and security
  Software dependability includes a range of characteristics including **reliability**, **security**, and **safety**. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.

- Efficiency
  Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilization, etc.

- Acceptability
  Software must be acceptable to the type of users for which it is designed. This means that it must be **understandable**, **usable**, and **compatible** with other systems that they use.

Software engineering

Software engineering is an **engineering discipline** that is concerned with all aspects of **software production** from the early stages of system specification through to maintaining the system after it has gone into use.

There are two key phrases in software engineering.

- Engineering discipline, Engineers make things work. They apply theories, methods, and tools where these are appropriate. However, they use them selectively and always try to discover solutions to problems even when there are no applicable theories and methods. Engineers also recognize that they must work to organizational and financial constraints, so they look for solutions within these constraints.

- All aspects of software production, Software engineering is not just concerned with the technical processes of software development. It also includes activities such as **software project management and the development of tools**, methods, and theories to support software production.

In software engineering, the software process (also known as the software development process or software lifecycle) refers to the structured set of activities, methods, and practices followed to develop, deliver, and maintain software systems. It defines how software projects are initiated, executed, and managed to ensure quality and efficiency.

Software Process

The **systematic approach** that is used in software engineering is sometimes called a software process. A software process is a **sequence of activities** that leads to the production of a software product.

Software process activities

1. **Software requirement specification (SRS)**, where customers and engineers define the software that is to be produced and the constraints on its operation.
2. **Software development**, where the software is designed and programmed.
3. **Software validation (Or Software Testing)**, where the software is checked to ensure that it is what the customer requires.
4. **Software evolution (Or Maintenance)**, where the software is modified to reflect changing customer and market requirements.

Software engineering is related to both computer science and systems engineering.

Interface College of Computer Applications (ICCA)

- Computer science is concerned with the theories and methods that underlie computers and software systems, whereas software engineering is concerned with the **practical problems** of producing software.

- System engineering is concerned with all aspects of the development and evolution of complex systems where software plays a major role. System engineering is therefore concerned with **hardware development, policy and process design** and system deployment, as well as software engineering.

Below are issues which affects the different types of software.

- Heterogeneity Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices. As well as running on general-purpose computers, software may also have to execute on mobile phones. We often have to integrate new software with older legacy systems written in different programming languages. The challenge here is to develop techniques for building dependable software that is flexible enough to cope with

this heterogeneity.

- **Business and social change:** Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software. Many traditional software engineering techniques are time consuming and delivery of new systems often takes longer than planned. They need to evolve so that the time required for software to deliver value to its customers is reduced.

- **Security and trust:** As software is intertwined with all aspects of our lives, it is essential that we can trust that software. This is especially true for remote software systems accessed through a web page or web service interface. We have to make sure that malicious users cannot attack our software and that information security is maintained.

Software engineering diversity

Software engineering results in producing **diversified applications** as below.

Stand-alone applications: These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network. Examples of such applications are office applications on a PC, CAD programs, photo manipulation software, etc.

Interactive transaction-based applications: These are applications that execute on a remote computer and that are accessed by users from their own PCs or terminals.

Ex: Web applications such as e-commerce applications where We can interact with a remote system to buy goods and services.

- Embedded control systems
  These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system. Examples of embedded systems include the software in a mobile (cell) phone, software that controls anti-lock braking in a car, and software in a microwave oven to control the cooking process.

Interface College of Computer Applications (ICCA)

- Batch processing systems
  These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs. Examples of batch systems include periodic billing systems, such as phone billing systems, and salary payment systems.

- Entertainment systems
  These are systems that are primarily for personal use and which are intended to entertain the user. Most of these systems are games of one kind or another. The quality of the user interaction offered is the most important distinguishing characteristic of entertainment systems.

- Systems for modelling and simulation
  These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects. These are often computationally intensive and require high-performance parallel systems for execution.

- Data collection systems
  These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
  The software has to interact with sensors and often is installed in a hostile environment such as inside an engine or in a remote location.

- Systems of systems
  These are systems that are composed of a number of other software systems. Some of these may be generic software products, such as a spreadsheet program. Other systems in the assembly may be specially written for that environment.

Software engineering ethics

Software engineering is carried out within a social and legal framework that limits the freedom of people working in that area. As a software engineer, we must accept that your job involves wider responsibilities than simply the application of technical skills. We must also behave in an ethical and morally responsible way if we are to be respected as a professional engineer.

Below are areas where standards of acceptable behavior are not bound by laws but by the more tenuous notion of professional responsibility. Some of these are as below.

- **Confidentiality,** We should normally respect the confidentiality of your employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

- **Competence,** We should not misrepresent your level of competence. We should not knowingly accept work that is outside your competence.

- **Intellectual property rights,** We should be aware of local laws governing the use of intellectual property such as patents and copyright. We should be careful to ensure that the intellectual property of employers and clients is protected.

- **Computer misuse,** We should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses or other malware).

Interface College of Computer Applications (ICCA)

Software processes

- A software process is **a set of related activities** that leads to the production of a software product.
- These activities may involve the development of software from scratch in a standard programming language like Java or C.
- However, business applications are not necessarily developed in this way. New business software is now often developed by extending and modifying existing systems or by configuring and integrating off-the-shelf software or system components.

There are many different software processes but all must include four activities that are fundamental to software engineering.

- **Software specification**: The functionality of the software and constraints on its operation must be defined.
- **Software design and implementation**: The software to meet the specification must be produced.
- **Software validation:** The software must be validated to ensure that it does what the customer wants.
- **Software evolution**: The software must evolve to meet changing customer needs.

Software processes include activities such as specifying a data model, designing a user interface, etc., and the ordering of these activities. However, as well as activities, process descriptions may also include below activities.

- **Products**, which are the outcomes of a process activity. For example, the outcome of the activity of architectural design may be a model of the software architecture.
- **Roles**, which reflect the responsibilities of the people involved in the process. Examples of roles are project manager, configuration manager, programmer, etc.
- **Pre- and post-conditions**, which are statements that are true before and after a process activity has been enacted or a product produced.

Software process models

A software process model is a **simplified representation** of a software process. Each process model represents a process from a particular perspective, and thus provides only partial information about that process.

Interface College of Computer Applications (ICCA)

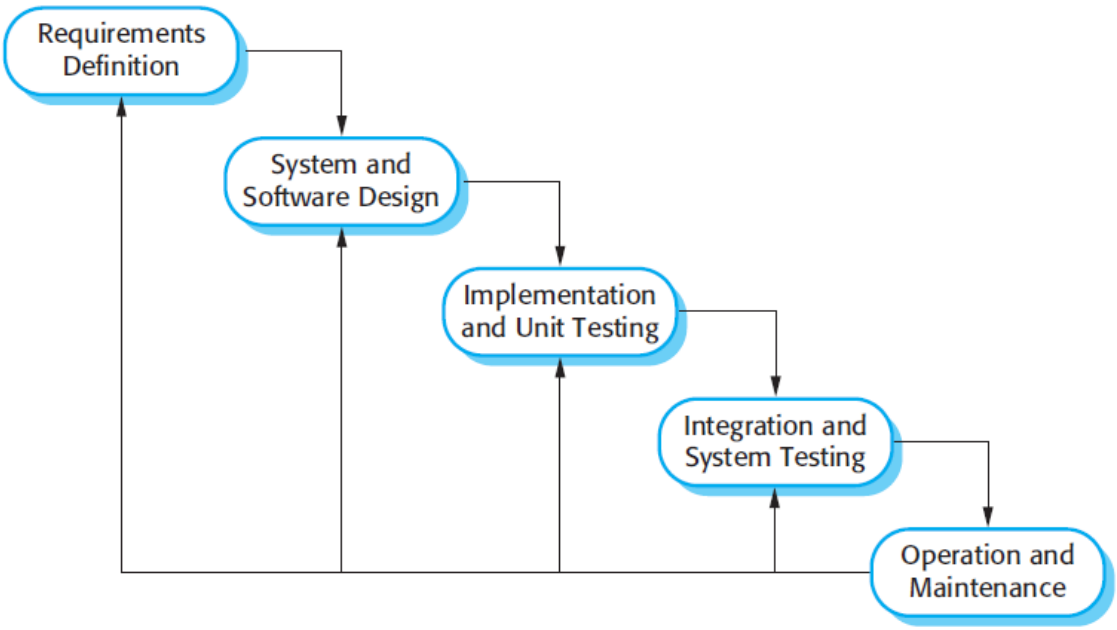
Note: A process activity model shows the activities and their sequence but may not show the roles of the people involved in these activities.

The generic process models are as below.

The waterfall models

- The first published model of the software development process was derived from more general system engineering processes (Royce, 1970).
- This model is illustrated in below figure.
- Because of the **cascade from one phase to another**, this model is known as the 'waterfall model' or software life cycle.
- The waterfall model is an example of a **plan-driven process**—in principle, We must plan and schedule all of the process activities before starting work on them.
- In principle, the result of each phase is one or more documents that are approved

- The following phase should not start until the previous phase has finished.



The principal stages of the waterfall model directly reflect the fundamental development activities.

- **Requirements analysis and definition:** The system's services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.
- **System and software design:** The systems design process allocates the requirements to either hardware or software systems by establishing an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.
- **Implementation and unit testing:** During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.
- **Integration and system testing:** The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
- **Operation and maintenance:** Normally (although not necessarily), this is the longest life cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

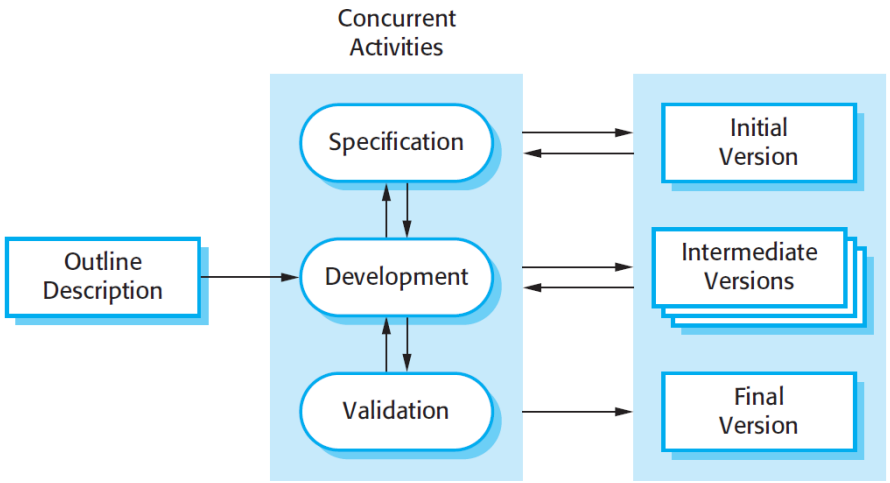
ICCA
Interface College of Computer Applications (ICCA)

Advantages and Disadvantages of Waterfall Model

Advantages	Disadvantages
▪ Relatively simple to understand. ▪ Each phase of development proceeds sequentially. ▪ Allows managerial control where a schedule with deadlines is set for each stage of development. ▪ Helps in controlling schedules, budgets, and documentation.	▪ Requirements need to be specified before the development proceeds. ▪ The changes of requirements in later phases of the waterfall model cannot be done. This implies that once an application is in the testing phase, it is difficult to incorporate changes at such a late phase. ▪ No user involvement and working version of the software is available when the software is developed. ▪ Does not involve risk management. ▪ Assumes that requirements are stable and are frozen across the project span.

Incremental development model

- Incremental development is based on the idea of developing **an initial implementation**, **exposing** this to user comment and **evolving** it through several versions until an adequate system has been developed in below figure.



- Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.
- Incremental software development, which is a **fundamental part of agile approaches**, is better than a waterfall approach for most business, e-commerce, and personal systems.
- Incremental development has three important benefits, compared to the waterfall model
    - The cost of accommodating changing customer requirements is reduced. The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
    - It is easier to get customer feedback on the development work that has been

done. Customers can comment on demonstrations of the software and see how much has been implemented. Customers find it difficult to judge progress from software design documents.

- More rapid delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included. Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Advantages and Disadvantages of Incremental development model

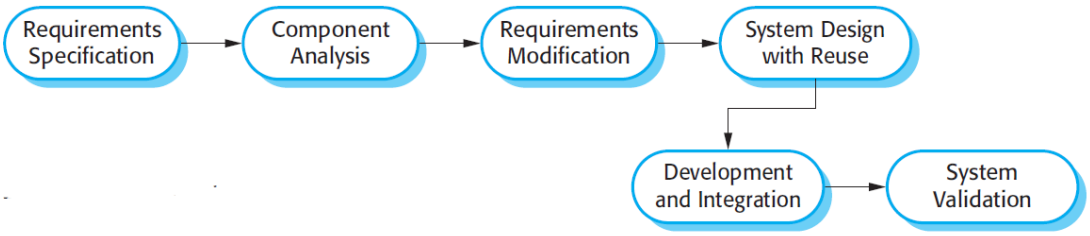
Advantages	Disadvantages
• The software will be generated quickly during the software life cycle	• It requires a good planning designing
• It is flexible and less expensive to change requirements and scope	• Problems might cause due to system architecture as such not all requirements collected up front for the entire software lifecycle
• Throughout the development stages changes can be done	• Each iteration phase is rigid and does not overlap each other
• This model is less costly compared to others	• Rectifying a problem in one unit requires correction in all the units and consumes a lot of time
• A customer can respond to each building	
• Errors are easy to be identified	

Reuse-oriented software engineering

- In most software projects, there is some software reuse. This often happens informally when people working on the project know of designs or code that are like what is required. They look for these, modify them as needed, and incorporate them into their system.
- This informal reuse takes place irrespective of the development process that is used.
- Reuse-oriented approaches rely on a large base of reusable software components and an integrating framework for the composition of these components. Sometimes, these

components are systems in their own right (COTS or commercial off-the-shelf systems) that may provide specific functionality such as word processing or a spreadsheet.

- A general process model for reuse-based development is shown in below figure.



- Although the initial requirements specification stage and the validation stage are comparable with other software processes, the intermediate stages in a reuse-oriented process are different.

These stages are as below.

- **Component analysis:** Given the requirements specification, a search is made for components to implement that specification. Usually, there is no exact match and the components that may be used only provide some of the functionality required.
- **Requirements modification:** During this stage, the requirements are analyzed using information about the components that have been discovered. They are then modified to reflect the available components. Where modifications are impossible, the component analysis activity may be re-entered to search for alternative solutions.
- **System design with reuse**: During this phase, the framework of the system is designed or an existing framework is reused. The designers take into account the components that are reused and organize the framework to cater for this. Some new software may have to be designed if reusable components are not available.
- **Development and integration:** Software, that cannot be externally procured is developed, and the components and COTS systems are integrated to create the new system. System integration, in this model, may be part of the development process rather than a separate activity.

Interface College of Computer Applications (ICCA)

There are three types of software component that may be used in a reuse-oriented Process.

- Web services, that are developed according to **service standards** and which are available for **remote invocation**.
- Collections of objects that are developed as a package to be integrated with a component framework such as .NET or J2EE.
- Stand-alone software systems that are configured for use in a particular environment.

Process activities

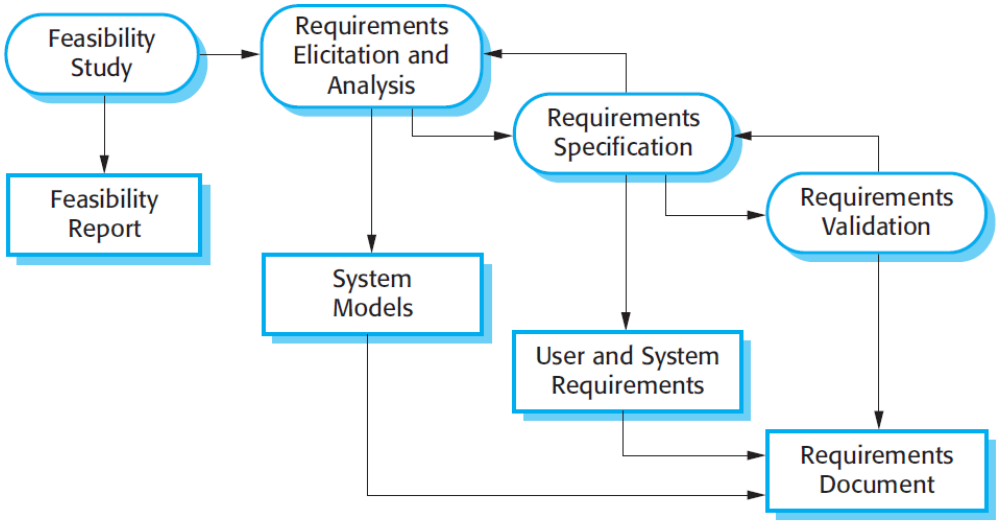
Real software processes are interleaved sequences of technical, collaborative, and managerial activities with the overall goal of **specifying, designing, implementing, and testing** a software system.

The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes.

Software specification

- Software specification or requirements engineering is the process of understanding and defining what services are required from the system and identifying the constraints on the system's operation and development.
- Requirements engineering is a particularly critical stage of the software process as errors at this stage **inevitably lead** to later problems in the system design and implementation.
- The requirements engineering process showed in below figure aims to produce an agreed requirements document that specifies a system satisfying stakeholder requirements.
- Requirements are usually presented at two levels of detail. End-users and customers need a high-level statement of the requirements; system developers need a more detailed system specification.

There are four main activities in the requirements engineering process.



- **Feasibility study**: An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints. A feasibility study should be relatively cheap and quick. The result should inform the decision of whether or not to go ahead with a more detailed analysis.
- **Requirements elicitation and analysis:** This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help We understand the system to be specified.
- **Requirements specification**: Requirements specification is the activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a

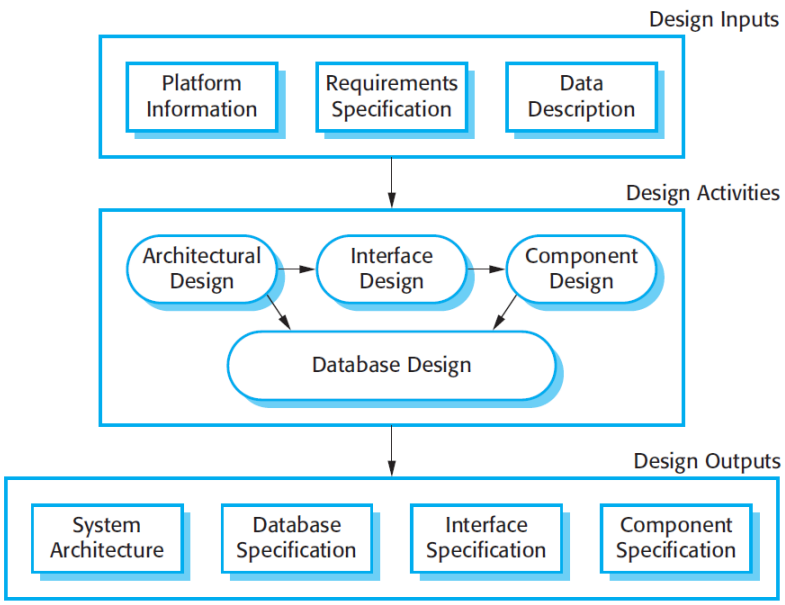
more detailed description of the functionality to be provided.

- **Requirements validation**: This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Software design and implementation

- The implementation stage of software development is the process of converting a system specification into an executable system. It always involves processes of software design and programming but, if an incremental approach to development is used, may also involve **refinement** of the software specification.
- A software design is a description of the structure of the software to be implemented, the data models and structures used by the system, the interfaces between system components and, sometimes, the algorithms used. Designers do not arrive at a finished design immediately but develop the design iteratively. They add formality and detail as they develop their design with constant backtracking to correct earlier designs.
- Below figure is an abstract model of this process showing the inputs to the design process, process activities, and the documents produced as outputs from this process.
- The diagram suggests that the stages of the design process are sequential. In fact, design process activities are **interleaved**. Feedback from one stage to another and consequent design rework is inevitable in all design processes.
- Most software interfaces with other software systems. These include the operating system, database, middleware, and other application systems. These make up the '**software platform'**, the environment in which the software will execute. Information about this platform is an essential input to the design process, as designers must decide how best to integrate it with the software's environment. The requirements specification is a description of the functionality the software must provide and its performance and dependability requirements. If the system is to process existing data, then the description of that data may be included in the platform specification; otherwise, the data description must be an input to the design process so that the system data organization to be defined.
- The activities in the design process vary, depending on the type of system being developed.

Interfa (ICCA)



- **Architectural design**, where We identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships, and how they are distributed.
- **Interface design**, where We define the interfaces between system components. This interface specification must be unambiguous. With a precise interface, a component can be used without other components having to know how it is implemented. Once interface specifications are agreed, the components can be designed and developed concurrently.
- **Component design**, where We take each system component and design how it will operate. This may be a simple statement of the expected functionality to be implemented, with the specific design left to the programmer. Alternatively, it may be a list of changes to be made to a reusable component or a detailed design model. The design model may be used to automatically generate an implementation.
- **Database design**, where we design the system data structures and how these are to be represented in a database. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

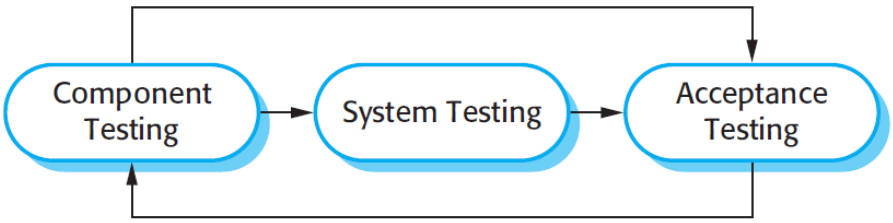
Note:

- Normally, programmers carry out some testing of the code they have developed.
- Programmers do some sort of testing as well as debugging.
- Testing establishes the existence of defects. Debugging is concerned with locating and correcting these defects.

Software validation

- Software validation or, more generally, verification and validation (V&V) is intended to show that a system both conforms to its specification and that it meets the expectations of the system customer.
- Program testing, where the system is executed using simulated test data, is the principal validation technique. Validation may also involve checking processes, such as inspections and reviews, at each stage of the software process from user requirements definition to program development.
- Because of the predominance of testing, the majority of validation costs are incurred during and after implementation.
- Below Figure shows a three-stage testing process in which system components are tested then the integrated system is tested and, finally, the system is tested with the customer's data. Ideally, component defects are discovered early in the process, and interface problems are found when the system is integrated.
- However, as defects are discovered, the program must be debugged and this may require other stages in the testing process to be repeated. Errors in program components, say, may come to light during system testing. The process is therefore an iterative one with information being fed back from later stages to earlier parts of the process.

Interface College of Computer Applications (ICCA)



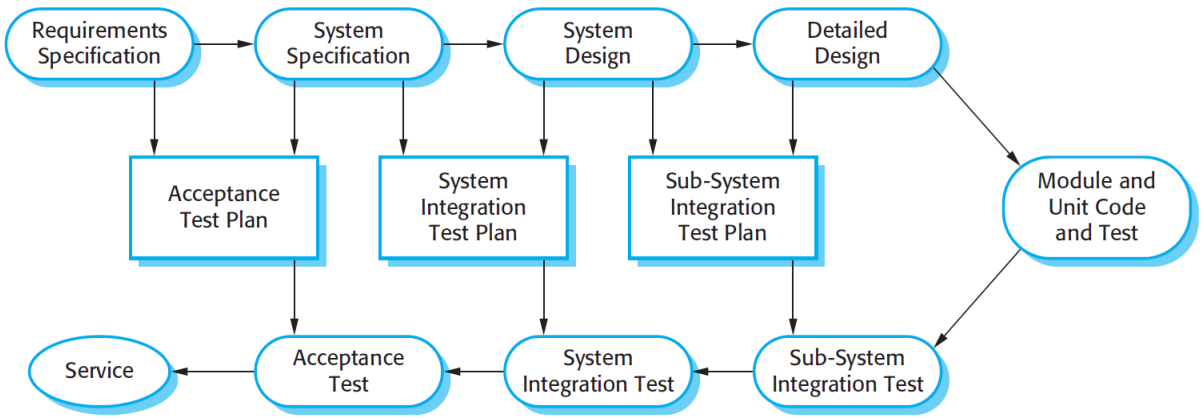
The stages in the testing process are

- **Development testing:** The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes, or may be coherent groupings of these entities. Test automation tools, such as JUnit (Massol and Husted, 2003), that can re-run component tests when new versions of the component are created, are commonly used.
- **System testing:** System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties. For large systems, this may be a multi-stage process where components are integrated to form subsystems that are individually tested before these sub-systems are themselves integrated to form the final system.
- **Acceptance testing** or **'alpha testing:** This is the final stage in the testing process before the system is accepted for operational use. The system is tested with data supplied by the system customer rather than with simulated test data. Acceptance testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data. Acceptance testing may also reveal requirements problems where the system's facilities do not really meet the user's needs or the system performance is unacceptable.

- When a system is to be marketed as a software product, a testing process called '**Beta testing'** is often used. Beta testing involves delivering a system to a number of potential customers who agree to use that system.

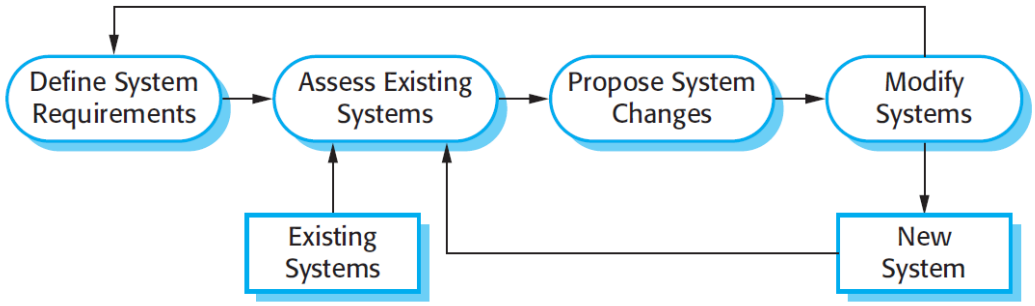
Interface College of Computer Applications (ICCA)

- Below figure illustrates how test plans are the link between testing and development activities and this is sometimes called the V-model of development.



Software evolution

- Software evolution nothing but software maintenance.
- Changes can be made to software at any time during or after the system development, but changes to hardware is not easy and affordable.
- Below figure shows the software evolution process where software is continually changed over its lifetime in response to changing requirements and customer needs.



Coping with change

- Change is **inevitable** in all large software projects.
- Changes in requirements, management priorities force to change the software with necessary modifications.
- Irrespective what software process model is used, it is essential that it can accommodate changes to the software being developed.
- Change adds to the costs of software development because it usually means that work that has been completed has to be redone. This is called **rework**.

There are two related approaches that may be used to reduce the costs of rework

- **Change avoidance**, where the software process includes activities that can anticipate possible changes before significant rework is required.
- **Change tolerance**, where the process is designed so that changes can be accommodated at relatively low cost. This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have to be altered to incorporate the change.

Interface College of Computer Applications (ICCA)

Coping with change and changing system requirements can be achieved through below approaches.

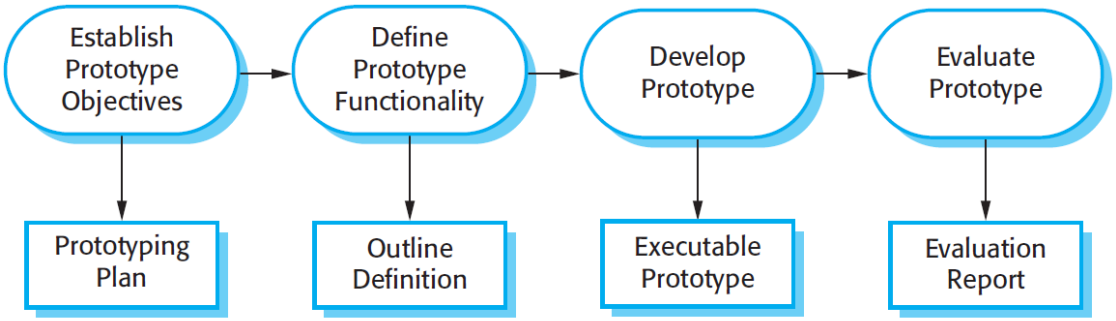
- **System prototyping**, where a version of the system or part of the system is developed quickly to check the customer's requirements and the feasibility of some design decisions. This supports change avoidance as it allows users to experiment with the system before delivery and so refine their requirements. The number of requirements change proposals made after delivery is therefore likely to be reduced.

- **Incremental delivery**, where system increments are delivered to the customer for comment and experimentation. This supports both change avoidance and change tolerance. It avoids the premature commitment to requirements for the

whole system and allows changes to be incorporated into later increments at relatively low cost.

Prototyping

- A prototype is an initial version of a software system that is used to demonstrate concepts, try out design options, and find out more about the problem and its possible solutions. Rapid, iterative development of the prototype is essential so that costs are controlled and system stakeholders can experiment with the prototype early in the software processes.
- A software prototype can be used in a software development process to help anticipate changes that may be required:
    - In the requirements engineering process, a prototype can help with the **elicitation and validation** of system requirements.
    - In the system design process, a prototype can be used to explore particular software solutions and to support user interface design.
- A process model for prototype development is shown in below figure

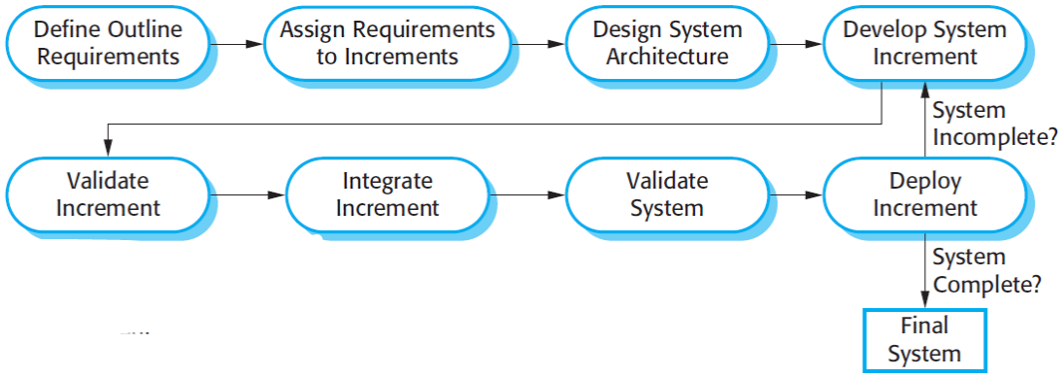


Advantages and Disadvantages of Prototyping Model

Advantages	Disadvantages
▪ Provides a working model to the user early in the process, enabling early assessment and increasing user confidence. ▪ Developer gains experience and insight by developing a prototype, thereby resulting in better implementation of requirements. ▪ Prototyping model serves to clarify requirements, which are not clear, hence reducing ambiguity and improving communication between developer and user. ▪ There is a great involvement of users in software development. Hence, the requirements of the users are met to the greatest extent. ▪ Helps in reducing risks associated with the project.	▪ If the user is not satisfied by the developed prototype, then a new prototype is developed. This process goes on until a perfect prototype is developed. Thus, this model is time consuming and expensive. ▪ Developer looses focus of the real purpose of prototype and compromise with the quality of the product. For example, they apply some of the inefficient algorithms or inappropriate programming languages used in developing the prototype. ▪ Prototyping can lead to false expectations. It often creates a situation where user believes that the development of the system is finished when it is not. ▪ The primary goal of prototyping is rapid development, thus, the design of system can suffer as it is built in a series of layers without considering integration of all the other components.

Incremental delivery

- It is an approach to software development where some of the developed increments are delivered to the customer and deployed for use in an operational environment and below figure elaborates the concept of incremental delivery.



- In an incremental delivery process, customers identify, in outline, the services to be provided by the system. They identify which of the services are most important and which are least important to them. A number of delivery increments are then defined, with each increment providing a sub-set of the system functionality.
- The allocation of services to increments depends on the service priority, with the highest-priority services implemented and delivered first.
- Once the system increments have been identified, the requirements for the services to be delivered in the first increment are defined in detail and that increment is developed. During development, further requirements analysis for later increments can take place but requirements change for the current increment are not accepted.
- Once an increment is completed and delivered, customers can put it into service. This means that they take early delivery of part of the system functionality. They can experiment with the system and this helps them clarify their requirements for later system increments. As new increments are completed, they are integrated with existing increments so that the system functionality improves with each delivered increment.

Interface College of Computer Applications (ICCA)

Incremental delivery has several advantages

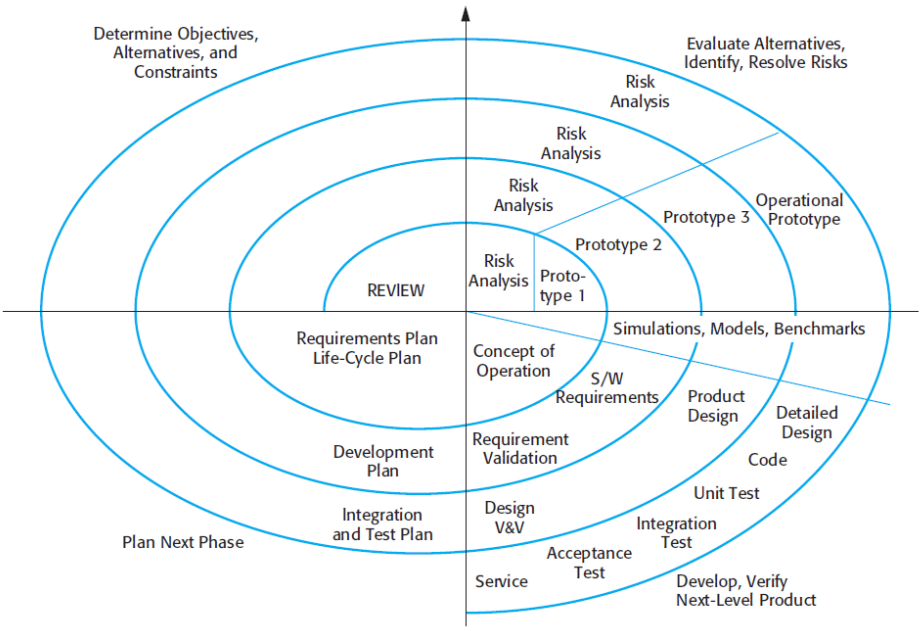
- Customers can use the early increments as prototypes and gain experience that informs their requirements for later system increments. Unlike prototypes, these are part of the real system so there is no re-learning when the complete system is available.
- Customers do not have to wait until the entire system is delivered before they can gain value from it. The first increment satisfies their most critical requirements so, they can use the software immediately.
- The process maintains the benefits of incremental development in that it should be relatively easy to incorporate changes into the system.
- As the highest-priority services are delivered first and increments then integrated, the most important system services receive the most testing. This means that customers are less likely to encounter software failures in the most important parts of the system.

However, there are problems with incremental delivery

- Most systems require a set of basic facilities that are used by different parts of the system. As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
- Iterative development can also be difficult when a replacement system is being developed. Users want all of the functionality of the old system and are often unwilling to experiment with an incomplete new system. Therefore, getting useful customer feedback is difficult.
- The essence of iterative processes is that the specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract. In the incremental approach, there is no complete system specification until the final increment is specified. This requires a new form of contract, which large customers such as government agencies may find difficult to accommodate.

Boehm's spiral model

- A risk-driven software process framework (the spiral model) was proposed by Boehm (1988).
- As shown below shown in figure. Here, the software process is represented **as a spiral**, **rather than a sequence of activities** with some backtracking from one activity to another.



- Each loop in the spiral represents a **phase of the software process**. Thus, the innermost loop might be concerned with system feasibility, the next loop with requirements definition, the next loop with system design, and so on.
- The spiral model combines **change avoidance with change tolerance**. It assumes that changes are a result of project risks and includes explicit risk management activities

to reduce these risks.

- Each loop in the spiral is split into four sectors:
  - **Objective setting**, Specific objectives for that phase of the project are defined. Constraints on the process and the product are identified and a detailed management plan is drawn up. Project risks are identified. Alternative strategies, depending on these risks, may be planned.
  - **Risk assessment and reduction**, For each of the identified project risks, a detailed analysis is carried out. Steps are taken to reduce the risk. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.
  - **Development and validation**, After risk evaluation, a development model for the system is chosen. For example, throwaway prototyping may be the best development approach if user interface risks are dominant. If safety risks are the main consideration, development based on formal transformations may be the most appropriate process, and so on. If the main identified risk is sub-system integration, the waterfall model may be the best development model to use.
  - **Planning,** The project is reviewed and a decision made whether to continue with a further loop of the spiral. If it is decided to continue, plans are drawn up for the next phase of the project.

Agile software development

Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently. It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

Agile Software Development is a software development methodology that **values flexibility, collaboration, and customer satisfaction.** It is based on the Agile Manifesto, a set of principles for software development that **prioritize individuals and interactions, working software, customer collaboration, and responding to change**.

Agile methods have been very successful for some types of system development

- **Product development**, where a software company is developing a small or medium-sized product for sale.
- **Custom system development within an organization**, where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.

Interface College of Computer Applications (ICCA)

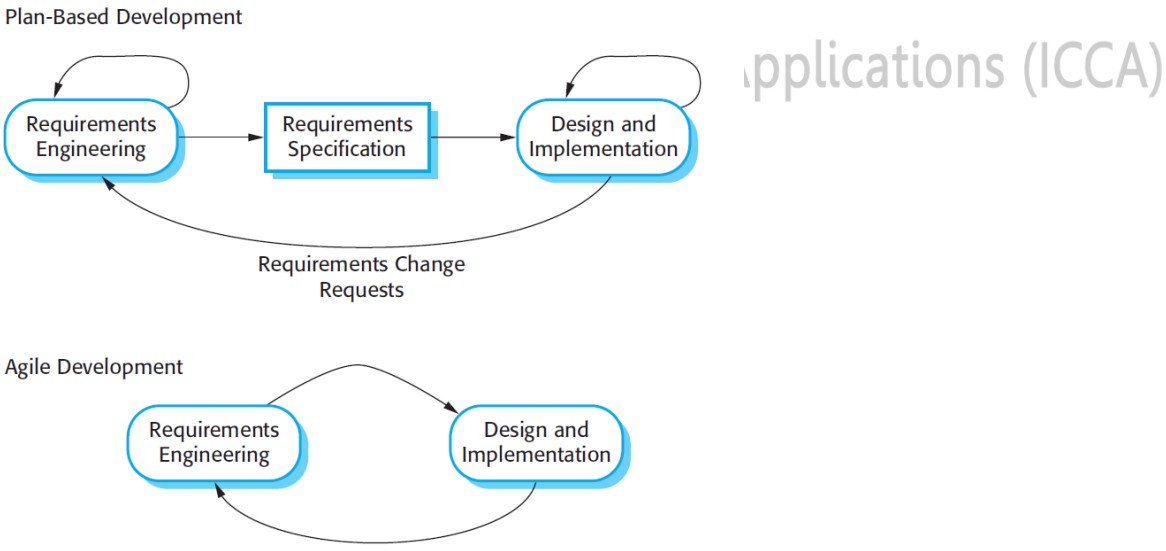
The principles of agile methods are as tabulated below.

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Plan-driven and agile development

- In a plan-driven approach, iteration occurs within activities with **formal documents** used to communicate between stages of the process.
- For example, the requirements will evolve and, ultimately, a requirements specification will be produced.
- This is then an input to the design and implementation process. In an agile approach, iteration occurs across activities. Therefore, the requirements and the design are developed together, rather than separately.

Below figure shows the distinctions between plan-driven and agile approaches to system specification.



- A plan-driven software process can support incremental development and delivery. It is perfectly feasible to allocate requirements and plan the design and development phase as a series of increments.
- An agile process is **not inevitably code-focused** and it may produce some design documentation.

Interface College of Computer Applications (ICCA)

UNIT-2: REQUIREMENTS ENGINEERING

Functional and non-functional requirements

Software system requirements are often classified as **functional requirements and nonfunctional requirements.**

Functional requirements

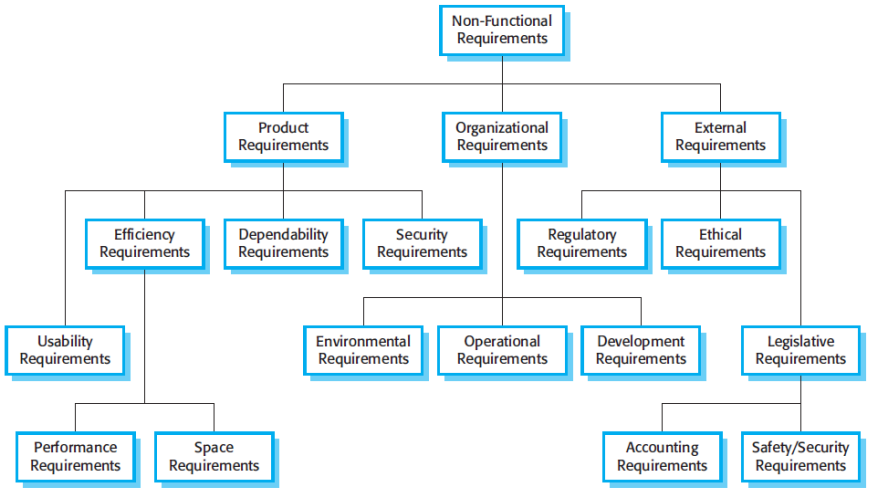
- The functional requirements for a system describe **what the system should do**.
- These are statements of services the system should provide, how the system should react to inputs, and how the system should behave situations. In some cases, the functional requirements may also explicitly state what the system should not do.
- The functional requirement of mental hospital is as below.
    - A user shall be able to search the appointments lists for all clinics.
    - The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
    - Each staff member using the system shall be uniquely identified by his or her eight-digit employee number.
- In principle, the functional requirements specification of a system should be both **complete and consistent**. Completeness means that all services required by the user should be defined. Consistency means that requirements should not have contradictory definitions.

Non-functional requirements

- Non-functional requirements, as the name suggests, are requirements that **are not directly concerned with the specific services** delivered by the system to its users.
- They may relate to emergent system properties such as reliability, response time, and store occupancy.
- These are constraints on the services or functions offered by the system. They include **timing constraints, constraints on the development process, and constraints imposed by standards.** Non-functional requirements often apply to the system as a whole, rather than individual system features or services.
- Non-functional requirements may affect the overall architecture of a system rather than the individual components.

Interface College of Computer Applications (ICCA)

Below figure shows the classification of non-functional requirements



- **Product requirements** These requirements specify or constrain the behaviour of the software. Examples include performance requirements on how fast the system must execute and how much memory it requires, reliability requirements that set out the acceptable failure rate, security requirements, and usability requirements.
- **Organizational requirements** These requirements are broad system requirements derived from policies and procedures in the customer's and developer's organization. Examples include operational process requirements that define how the system will be used, development process requirements that specify the programming language, the development environment or process standards to be used, and environmental requirements that specify the operating environment of the system.
- **External requirements** This broad heading covers all requirements that are derived from factors external to the system and its development process. These may include regulatory requirements that set out what must be done for the systemto be approved for use by a regulator, such as a central bank; legislative requirements that must be followed to ensure that the system operates within the law; and ethical requirements that ensure that the system will be acceptable to its users and the general public.

Interface College of Computer Applications (ICCA)

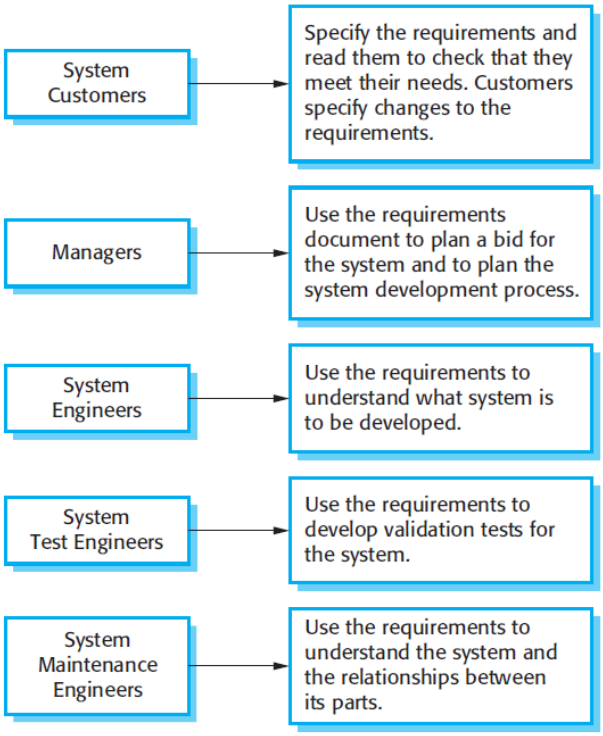
Metrics for specifying non-functional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

The software requirements document

- The software requirements document (sometimes called the software requirements specification or SRS) is an official statement of **what the system developers should implement**. It should include both the user requirements for a system and a detailed specification of the system requirements.
- Sometimes, the user and system requirements are integrated into a single description. In other cases, the user requirements are defined in an introduction to the system requirements specification. If there are many requirements, the detailed system requirements may be presented in a separate document.
- Most of the time agile development don't need SRS.
- The requirements document has a diverse set of users, ranging from the senior management of the organization that is paying for the system to the engineers responsible for developing the software as shown in below figure.
- The diversity of possible users means that the requirements document has to be a compromise between communicating the requirements to customers, defining the requirements in precise detail for developers and testers, and including information about possible system evolution.

Interface College of Computer Applications (ICCA)



ICCA
Interface College of Computer Applications (ICCA)

- Below figure shows one possible organization for a requirements document that is based on an IEEE standard for requirements documents (IEEE, 1998). This standard is a generic standard that can be adapted to specific uses.

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The non-functional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System requirements specification	This should describe the functional and non-functional requirements in more detail. If necessary, further detail may also be added to the non-functional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components, the system, and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Note:

The flexibility of natural language, which is so useful for specification, often causes problems. There is scope for writing unclear requirements, and readers (the designers) may misinterpret requirements because they have a different background to the user. It is easy to **amalgamate**(combine) several requirements into a single sentence and structuring natural language requirements can be difficult.

Requirements specification

- Requirements specification is the process of writing down the user and system requirements in a requirements document. Ideally, the user and system requirements **should be clear, unambiguous, easy to understand, complete, and consistent**.
- The user requirements for a system should describe the functional and nonfunctional requirements so that they are understandable by system users who don't have detailed technical knowledge.
- System requirements are expanded versions of the user requirements that are used by software engineers as the starting point for the system design. They add detail and explain how the user requirements should be provided by the system.
- Ideally, the system requirements should simply describe the external behaviour of the system and its operational constraints.
- User requirements are almost always written in natural language supplemented by appropriate diagrams and tables in the requirements document. System requirements may also be written in natural language but other notations based on forms, graphical system models, or mathematical system models can also be used.

Below are ways of writing a system requirement specification.

Notation	Description
Natural language sentences	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract.

Natural language specification

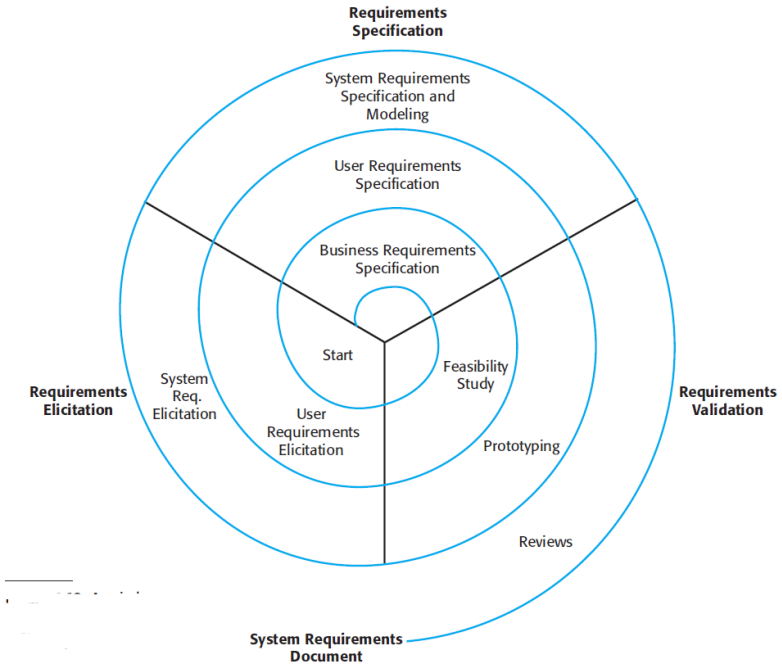
- Natural language has been used to write requirements for software since the beginning of software engineering. It is **expressive, intuitive, and universal**. It is **also potentially vague, ambiguous,** and its meaning depends on the background of the reader. As a result, there have been many proposals for alternative ways to write requirements.
- However, none of these have been widely adopted and natural language will continue to be the most widely used way of specifying system and software requirements.

Structured specifications

- Structured natural language is a way of writing system requirements where, the freedom of the requirements writer is limited and all requirements are written in a standard way. This approach maintains most of the expressiveness and understandability of natural language but ensures that some uniformity is imposed on the specification.
- Structured language notations use templates to specify system requirements. The specification may use programming language constructs to show alternatives and iteration, and may highlight key elements using shading or different fonts.

Requirements engineering processes.

- Requirements engineering processes may include four **high-level** activities. These focus on assessing if the system is useful to the business (**feasibility study**), discovering requirements (**elicitation and analysis**), converting these requirements into some standard form (**specification**), and checking that the requirements actually define the system that the customer wants **(validation**).
- I have shown these as sequential processes in below figure. However, in practice, requirements engineering is an iterative process in which the activities are interleaved.
- The activities are **organized as an iterative process around a spiral**, with the output being a system requirements document.
- The amount of time and effort devoted to each activity in each iteration depends on the stage of the overall process and the type of system being developed. Early in the process, most effort will be spent on understanding high-level business and non-functional requirements, and the user requirements for the system. Later in the process, in the outer rings of the spiral, more effort will be devoted to eliciting and understanding the detailed system requirements.

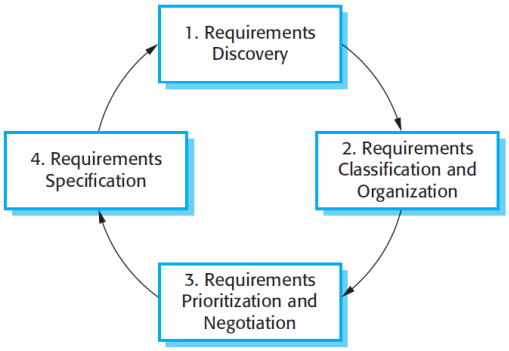


- This spiral model accommodates approaches to development where the requirements are developed to different levels of detail. The number of iterations around the spiral can vary so the spiral can be exited after some or all of the user requirements have been elicited. Agile development can be used instead of

prototyping so that the requirements and the system implementation are developed together.

Requirement's elicitation and analysis

- After an initial feasibility study, the next stage of the requirements engineering process is requirements elicitation and analysis. In this activity, software engineers work with customers and system end-users to **find out about the application domain, what services the system should provide, the required performance of the system, hardware constraints, and so on.**
- Requirements elicitation and analysis may involve a variety of different kinds of people in an organization. **A system stakeholder** is anyone who should have some direct or indirect influence on the system requirements. Stakeholders include end users who will interact with the system and anyone else in an organization who will be affected by it. Other system stakeholders might be engineers who are developing or maintaining other related systems, business managers, domain experts, and trade union representatives.
- Below figure shows process activities.



- **Requirements discovery:** This is the process of interacting with stakeholders of the system to discover their requirements. Domain requirements from stakeholders and documentation is also discovered during this activity. There are several complementary techniques that can be used for requirements discovery, which I discuss later in this section.
- **Requirements classification and organization:** This activity takes the unstructured collection of requirements, groups related requirements, and organizes them into coherent clusters. The most common way of grouping requirements is to use a model of the system architecture to identify sub-systems and to associate requirements with each sub-system. In practice, requirements engineering and architectural design cannot be completely separate activities.
- **Requirements prioritization and negotiation:** Inevitably, when multiple stakeholders are involved, requirements will conflict. This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation. Usually, stakeholders have to meet to resolve differences and agree on compromise requirements.
- **Requirements specification:** The requirements are documented and input into the next round of the spiral. Formal or informal requirements documents may be produced, as discussed in above section.

Interface College of Computer Applications (ICCA)

Requirements discovery

- Requirements discovery (sometime called requirements elicitation) is **the process of gathering information about the required system and existing systems and distilling the user and system requirements from this information.**
- Sources of information during the requirements discovery phase include documentation, system stakeholders, and specifications of similar systems.
- Different viewpoints on a problem see the problem in different ways. However, their perspectives are not completely independent but usually overlap so that they have common requirements. We can use these viewpoints to structure both the discovery and the documentation of the system requirements.
- Stakeholders range from end-users of a system through managers to external stakeholders such as regulators, who certify the acceptability of the system.
- In addition to system stakeholders, we have already seen that requirements may also come from the application domain and from other systems that interact with the system being specified. All these viewpoints must be considered during the requirements elicitation process.

Interviewing

- Formal or informal interviews with system stakeholders are part of most requirements engineering processes. In these interviews, the requirements engineering team puts questions to stakeholders about the system that they currently use and the system to be developed.
- Requirements are derived from the answers to these questions.
- Interviews may be of two types.
  - **Closed interviews**, where the stakeholder answers a pre-defined set of questions. **Open interviews**, in which there is no pre-defined agenda. The requirements engineering team explores a range of issues with system stakeholders and hence develop a better understanding of their needs.

To overcome the drawbacks of interviews following characteristics needs to follow.

- They are open-minded, avoid pre-conceived ideas about the requirements, and are willing to listen to stakeholders. If the stakeholder comes up with surprising requirements, then they are willing to change their mind about the system.
- They prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system. Saying to people 'tell me what you want' is unlikely to result in useful information. They find it much easier to talk in a defined context rather than in general terms.

Interface College of Computer Applications (ICCA)

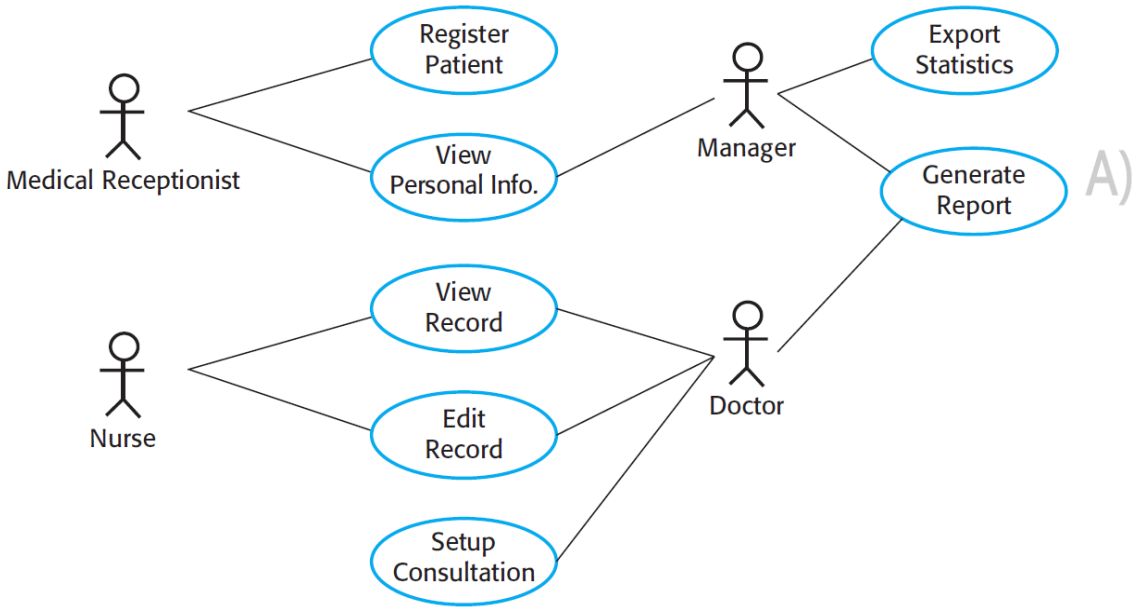
Scenarios

- People usually find it easier to relate to real-life examples rather than abstract descriptions. They can understand and criticize a scenario of how they might interact with a software system.
- Requirements engineers can use the information gained from this discussion to formulate the actual system requirements.
- Scenarios can be particularly useful for adding detail to an outline requirements description. They are descriptions of example interaction sessions. Each scenario usually covers one or a small number of possible interactions.
- Different forms of scenarios are developed, and they provide different types of information at different levels of detail about the system.

- A scenario starts with an outline of the interaction. During the elicitation process, details are added to this to create a complete description of that interaction. At its most general, a scenario may include.
  - A description of what the system and users expect when the scenario starts.
  - A description of the normal flow of events in the scenario.
  - A description of what can go wrong and how this is handled.
  - Information about other activities that might be going on at the same time.
  - A description of the system state when the scenario finishes.

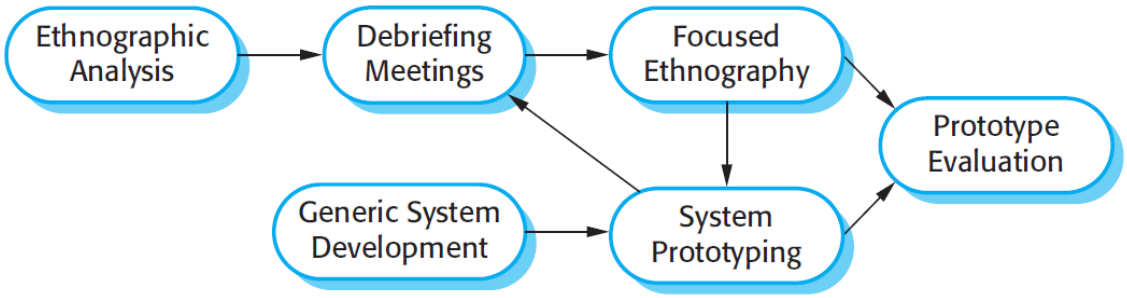
Use cases

- Use cases are a requirements discovery technique that were first introduced in the objectory method.
- They have now become a fundamental feature of the unified modelling language. In their simplest form, **a use case identifies the actors involved in an interaction and names the type of interaction.** This is then supplemented by additional information describing the interaction with the system.
- The additional information may be a textual description or one or more graphical models such as UML sequence or state charts.
- Actors in the process, who may be human or other systems, are represented as stick figures. **Each class of interaction is represented as a named ellipse. Lines link the actors with the interaction. Optionally, arrowheads may be added to lines to show how the interaction is initiated**.
- Below use case diagram shows the sample interactions of actors in hospital system.



Ethnography

- Ethnography is an **observational technique** that can be used to **understand operational processes** and help derive support requirements for these processes.
- An analyst immerses himself or herself in the working environment where the system will be used. The day-to-day work is observed, and notes made of the actual tasks in which participants are involved.
- The value of ethnography is that it helps discover implicit system requirements that reflect the actual ways that people work, rather than the formal processes defined by the organization.
- Below figure shows Ethnography and prototyping for requirements analysis



- Ethnography is particularly effective for discovering two types of requirements.
  - Requirements that are derived from the way in which people work, rather than the way in which process definitions say they ought to work.
  - Requirements that are derived from cooperation and awareness of other people's activities.

Requirements validation

- Requirements validation is the process **of checking that requirements actually define the system that the customer really wants.** It overlaps with analysis as it is concerned with finding problems with the requirements.
- Requirements validation is important because errors in a requirements document can **lead to extensive rework costs** when these problems are discovered during development or after the system is in service.
- The cost of fixing a requirements problem by making a system change is usually much greater than repairing design or coding errors. The reason for this is that a change to the requirements usually means that the system design and implementation must also be changed.
- Furthermore, the system must then be re-tested. During the requirements validation process, different types of checks should be carried out on the requirements in the requirements document. These checks as follows.
  - **Validity checks:** A user may think that a system is needed to perform certain functions. However, further thought and analysis may identify additional or different functions that are required. Systems have diverse stakeholders with different needs and any set of requirements is inevitably a compromise across the stakeholder community.
  - **Consistency checks:** Requirements in the document should not conflict. That is, there should not be contradictory constraints or different descriptions of the same system function.

ICCA Interface College of Computer Applications (ICCA)

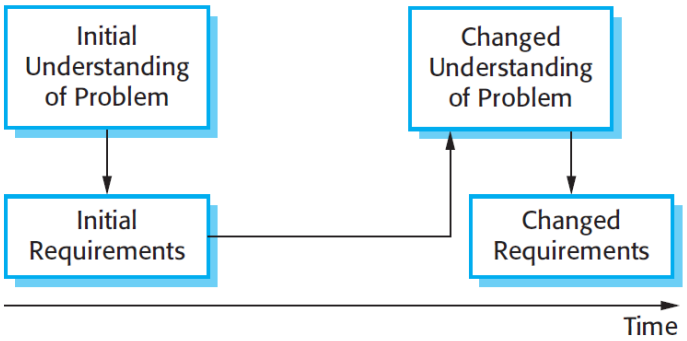
- **Completeness checks:** The requirements document should include requirements that define all functions and the constraints intended by the system user.
- **Realism checks:** Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented. These checks should also take account of the budget and schedule for the system development.
- **Verifiability:** To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. This means that you should be able to write a set of tests that can demonstrate that the delivered system meets each specified requirement.

- There are several requirements validation techniques that can be used individually or in conjunction with one another.
  - **Requirements reviews:** The requirements are analysed systematically by a team of reviewers who check for errors and inconsistencies.
  - **Prototyping:** In this approach to validation, an executable model of the system in question is demonstrated to end-users and customers. They can experiment with this model to see if it meets their real needs.
  - **Test-case generation:** Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems.

Requirements management

- The requirements for large software systems are always changing. One reason for this is that these systems are usually developed to address **'wicked' problems**—problems that cannot be completely defined.
- Because the problem cannot be fully defined, the software requirements are bound to be incomplete.
- During the software process, the stakeholders' understanding of the problem is constantly changing as shown below figure.

ICCA
Interfa ions (ICCA)



- The system requirements must then also evolve to reflect this changed problem view.
- Once a system has been installed and is regularly used, new requirements inevitably emerge. It is hard for users and system customers to anticipate what effects the new system will have on their business processes and the way that work is done. Once end users have experience of a system, they will discover new needs and priorities.
- Requirements management is the process of **understanding and controlling changes** to system requirements. We need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. We need to establish a formal process for making change proposals and linking these to system requirements. The formal process of requirements

management should start as soon as a draft version of the requirements document is available. However, you should start planning how to manage changing requirements during the requirements elicitation process.

Requirements management planning

Planning is an essential first stage in the requirements management process. The planning stage establishes the level of requirements management detail that is required. During the requirements management stage, we must decide on

- **Requirements identification:** Each requirement must be uniquely identified so that it can be cross-referenced with other requirements and used in traceability assessments.
- **A change management process:** This is the set of activities that assess the impact and cost of changes.
- **Traceability policies:** These policies define the relationships between each requirement and between the requirements and the system design that should be recorded. The traceability policy should also define how these records should be maintained.
- **Tool support Requirements:** management involves the processing of large amounts of information about the requirements. Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

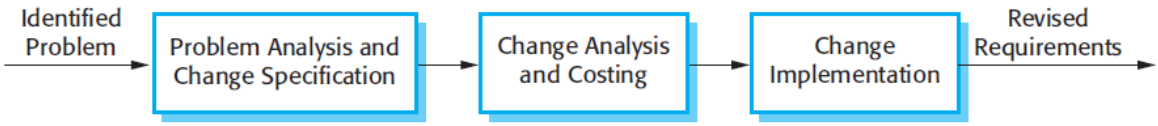
Requirements management needs automated support and the software tools for this should be chosen during the planning phase. We need tool support for

- **Requirements storage:** The requirements should be maintained in a secure, managed data store that is accessible to everyone involved in the requirements engineering process.
- **Change management:** The process of change management (as shown in below figure) is simplified if active tool support is available.
- **Traceability management:** As discussed above, tool support for traceability allows related requirements to be discovered. Some tools are available which use natural language processing techniques to help discover possible relationships between requirements.

Interface College of Computer Applications (ICCA)

Requirements change management.

- Requirements change management should be applied to all proposed changes to a system's requirements after the requirements document has been approved.
- Change management is essential because you need to decide if the benefits of implementing new requirements are justified by the costs of implementation. The advantage of using a formal process for change management is that all change proposals are treated consistently and changes to the requirements document are made in a controlled way.
- There are three principal stages to a change management process as shown in below figure



- **Problem analysis and change specification:** The process starts with an identified requirements problem or, sometimes, with a specific change proposal. During this stage, the problem or the change proposal is analysed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.
- **Change analysis and costing:** The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost of making the change is estimated both in terms of modifications to the requirements document and, if appropriate, to the system design and implementation. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
- **Change implementation:** The requirements document and, where necessary, the system design and implementation, are modified. You should organize the requirements document so that you can make changes to it without extensive rewriting or reorganization. As with programs, changeability in documents is achieved by minimizing external references and making the document sections as modular as possible. Thus, individual sections can be changed and replaced without affecting other parts of the document.

ICCA
Interface College of Computer Applications (ICCA)

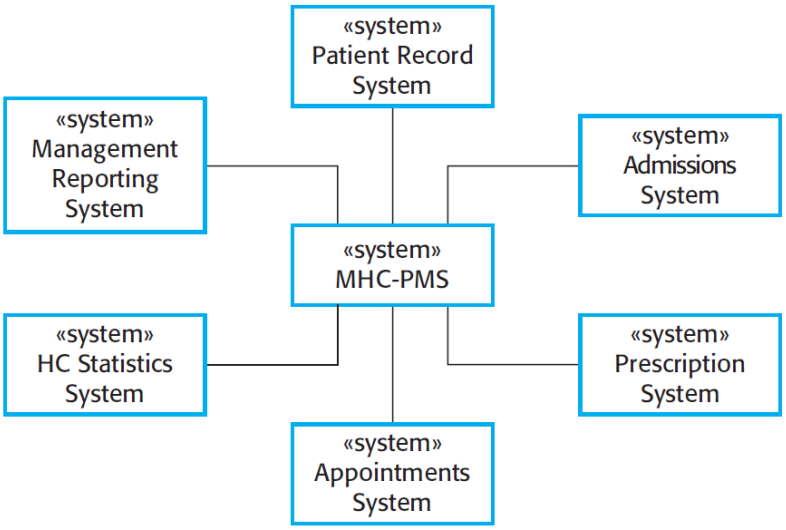
UNIT-3: SYSTEM MODELING

- System modelling is the process of **developing abstract models** of a system, with each model presenting a **different view or perspective** of that system.
- System modelling has generally come to mean representing the system using some kind of **graphical notation**, which is now almost always based on notations in the Unified Modelling Language (UML).
- However, it is also possible to develop **formal (mathematical) models** of a system, usually as a detailed system specification.
- Models are used during the **requirements engineering process** to help derive the requirements for a system, during the design process to describe the system to engineers implementing the system and after implementation to document the system's structure and operation. We may develop models of both the existing system and the system to be developed.
  - Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
  - Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation. In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

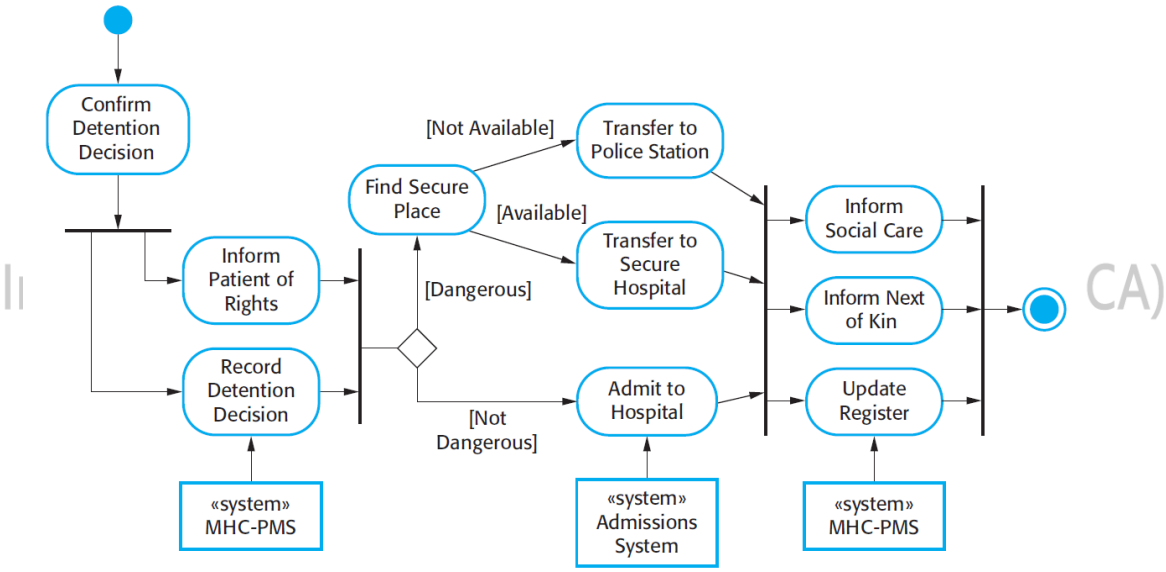
Context models

- Context models are simple **communication tools used to depict (**represent by a drawing) **the context of a business, a system, or a process**. The context is the environment in which the object of our interest exists.
- Context models capture how the central **object interacts with its environment**, be it exchanging data, physical objects, or funds.
- Below figure shows is a simple context model that shows the patient information system and the other systems in its environment and we can see that the MHC-PMS is connected to an appointments system and a more general patient record system with which it shares data. The system is also connected to systems for management reporting and hospital bed allocation and a statistics system that collects information for research. Finally, it makes use of a prescription system to generate prescriptions for patients' medication.

Interface College of Computer Applications (ICCA)



- Sometimes simple context models are used along with other models, such as **business process models**. These describe **human and automated** processes in which particular software systems are used.
- Below system process that shows the processes in which the MHC-PMS is used. Sometimes, patients who are suffering from mental health problems may be a danger to others or to themselves. They may therefore have to be detained against their will in a hospital so that treatment can be administered.

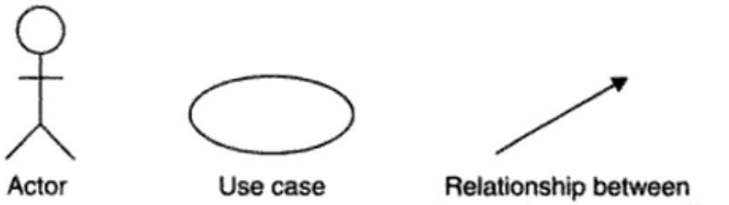


Interaction models

- All systems **involve interaction** of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the system being developed and other systems or interaction between the components of the system.
- **Modelling user interaction is important as it helps to identify user requirements**. Modelling system to system interaction highlights the **communication problems** that may arise.
- Modelling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.
- The related approaches of interaction modelling are as follows
  - **Use case modelling**, which is mostly used to model interactions between a system and external actors (users or other systems).
  - **Sequence diagrams**, which are used to model interactions between system components, although external agents may also be included.

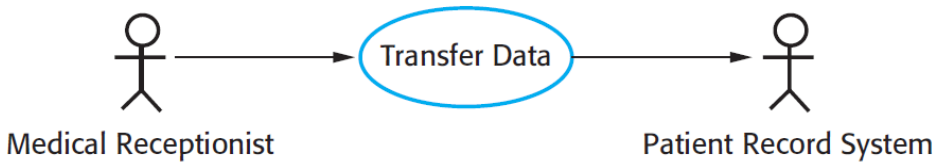
Use case modelling

- A use-case model is a model of **how different types of users interact with the system** to solve a problem. As such, it describes the goals of the users, the interactions between the users and the system, and the required behaviour of the system in satisfying these goals.
- A use case can be taken as a simple scenario that describes what a user expects from a system.
- Each use case represents a discrete task that involves external interaction with a system. In its simplest form, a use case is shown as an ellipse with the actors involved in the use case represented as stick figures.



Components of use case diagram.

- Below figure shows a use case from the MHC-PMS that represents the task of uploading data from the MHC-PMS to a more general patient record system.

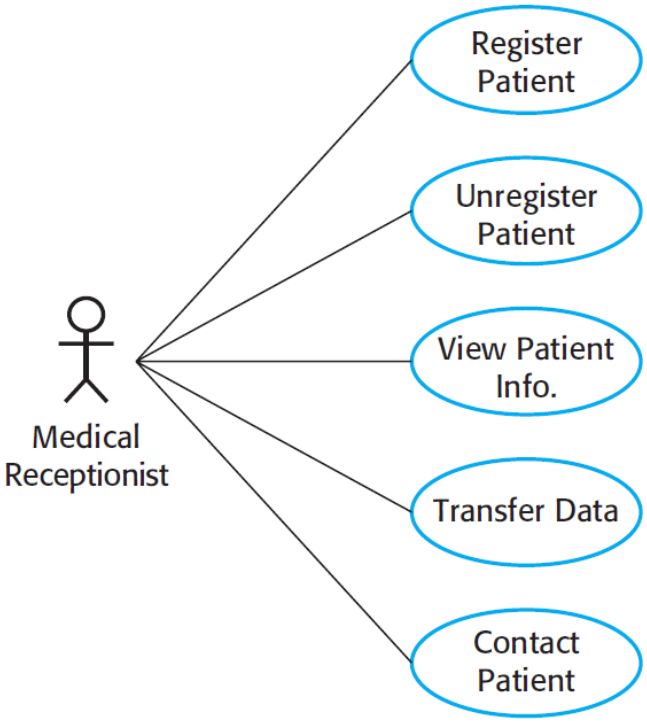


- Notice that there are two actors in this use case: the operator who is transferring the data and the patient record system. The stick figure notation was originally developed to cover human interaction, but it is also now used to represent other external systems and hardware.

- Below figure shows a tabular description of the 'Transfer data' use case.

MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

- Below figure shows all the use cases in the MHC-PMS in which the actor 'Medical Receptionist' is involved.



Sequence diagram

- A sequence diagram is one of the multiple types of system interaction diagrams to visually represent interactions **between the objects that live** within a system.
- Specifically, sequence diagrams provide a view of the order in which those interactions occur through depictions of individual objects, called lifelines, and the messages between them. With proper notation, teams can use sequence diagrams in multiple stages of the development process to map out the timing of interactions.

Sequence diagram notations

There are many components of a sequence diagram, which consists of various standardized dimensions and notations.

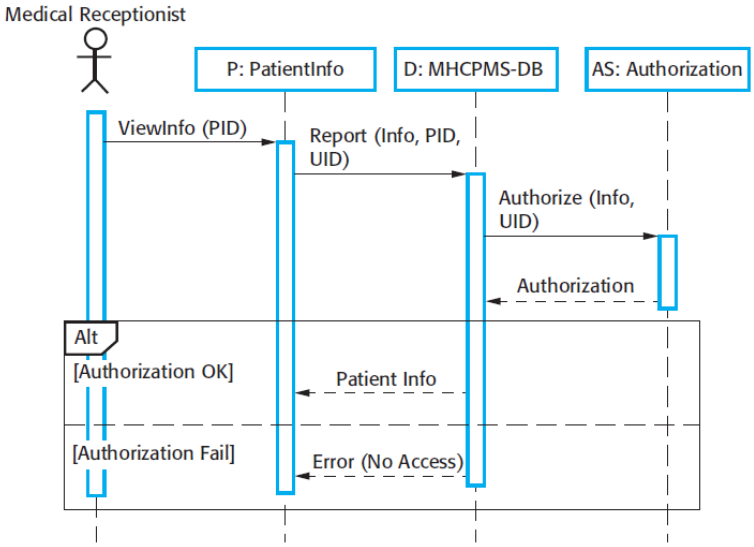
- The two major axis dimensions are objects and time. Objects, which run across the **horizontal axis**, show the various elements that the interactions revolve around. The **vertical axis**, time, indicates the sequential progression of the interactions depicted in the diagram -- note, however, that it does not indicate duration of those interactions.
- Within those dimensions, sequence diagrams typically include the following major notations.

  - **Lifelines** represent an individual participant within an interaction. Lifelines are represented by vertical dashed lines, connecting at the top to an object or actor.
  - **Actors represent the user**, external hardware or other subject setting off a particular sequence. They often appear as a stick figure icon and are placed to the left of the first object on the horizonal axis.
  - **Activation bars** -- sometimes called focus of control -- are thin rectangular boxes that sit vertically on top of an object's lifeline to indicate the duration of a particular operation.
  - **Interaction fragments** have no general notation but are used to describe an element of interaction in a sequence diagram. Examples of interaction types include occurrence, execution, state invariant, interaction use and combined fragment.
  - **Guards,** also known as interaction constraints, are square brackets positioned around an interaction as a Boolean expression. They represent a check or pause to make sure certain conditions have been met.
  - **Messages** represent communication from one actor or object to another. The first message typically starts at the top left of the sequence diagram and moves horizontally across lifelines from left to right, with each subsequent message one step lower than the one before.

  Below figure shows the sequence diagram for view patient information and it convey below points

  - The medical receptionist triggers the View Info method in an instance P of the Patient Info object class, supplying the patient's identifier, PID. P is a user interface object, which is displayed as a form showing patient information.
  - The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking (at this stage, we do not care where this UID comes from).
  - The database checks with an authorization system that the user is authorized for this action.
  - If authorized, the patient information is returned and a form on the user's screen is filled in. If authorization fails, then an error message is returned.

Interface College of Computer Applications (ICCA)



Structural models

- Structural models of software display the organization of a system in terms of the **components** that make up that **system** and their **relationships**.
- Structural models may be **static models**, which show the structure of the system design or dynamic models, which show the organization of the system when it is executing.
- We create structural models of a system when you are discussing and designing the system architecture.

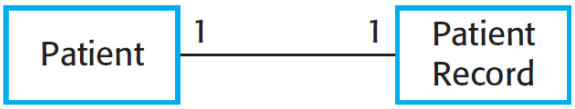
Note

In object-oriented requirements analysis, we model real-world entities using object classes. We may create different types of object model, showing how object classes are related to each other, how objects are aggregated to form other objects, how objects interact with other objects, and so on. These each present unique information about the system that is being specified.

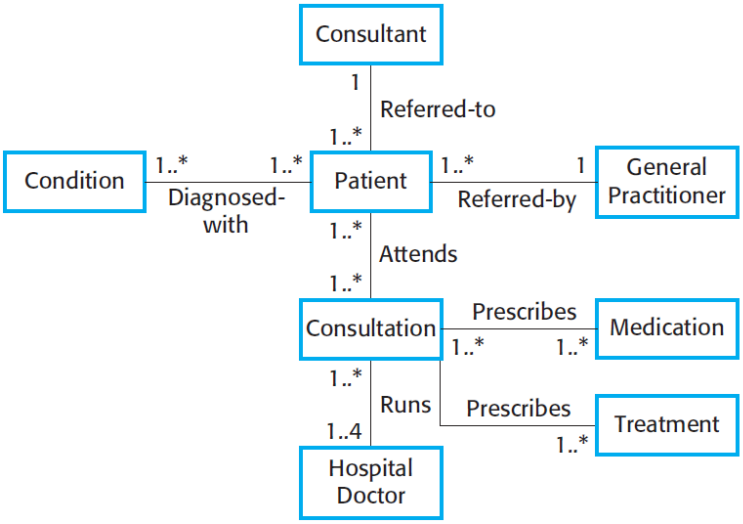
Interface College of Computer Applications (ICCA)

Class diagrams

- Class diagrams are used when developing an **object-oriented system model** to show the classes in a system and the associations between these classes.
- Loosely, an object class can be thought of as a general definition of one kind of system object.
- An association is a link between classes that indicates that there is a relationship between these classes. Consequently, each class may have to have some knowledge of its associated class.
- Class diagrams in the UML can be expressed at different levels of detail. When we are developing a model, the first stage is usually to look at the world, identify the essential objects and represent these as classes. The simplest way of writing these is to write the class name in a box. You can also simply note the existence of an association by drawing a line between classes as shown in below figure. It is a simple class diagram showing two classes: Patient and Patient Record with an association between them.

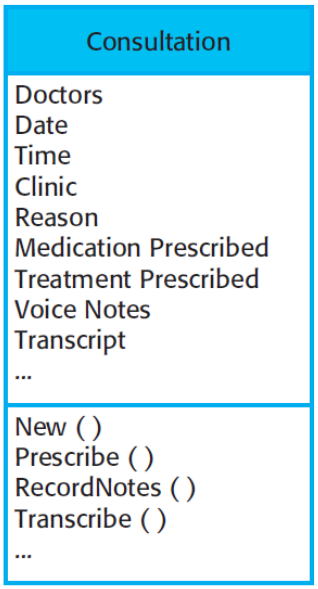


Below figure shows the class diagram of MHC-PMS and it convey the cardinality rations among the entities.



Below figure shows the detailed view of consolation class with its attributes and functionalities

Interface College of Computer Applications (ICCA)

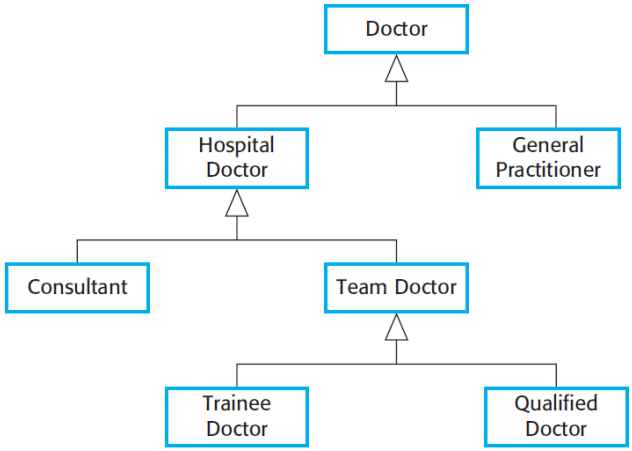


Above class diagram can be explained as below

- The name of the object class is in the top section.
- The class attributes are in the middle section. This must include the attribute names and, optionally, their types.
- The operations (called methods in Java and other OO programming languages) associated with the object class are in the lower section of the rectangle.

Generalization

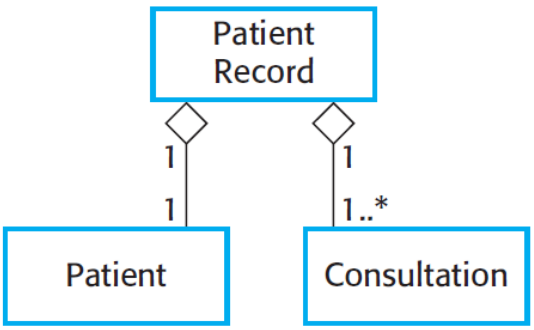
- Generalization is a process of extracting shared characteristics from two or more objects/processes and combine them into generalizing objects/ processes.
- This high-level concept is being used everywhere for different purposes in traditional software engineering practices.
- In modelling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. This means that common information will be maintained in one place only.
- In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes. In essence, the lower-level classes are subclasses inherit the attributes and operations from their superclasses.
- These lower-level classes then add more specific attributes and operations.
- The generalization is shown as an arrowhead pointing up to the more general class as shown in below figure.



Interface ons (ICCA)

Aggregation

- The UML provides a special type of association between classes called aggregation that means that one object (the whole) is composed of other objects (the parts). To show this, we use a diamond shape next to the class that represents the whole.
- This is shown in below figure, which shows that a patient record is a composition of Patient and an indefinite number of Consultations.



Behavioural models

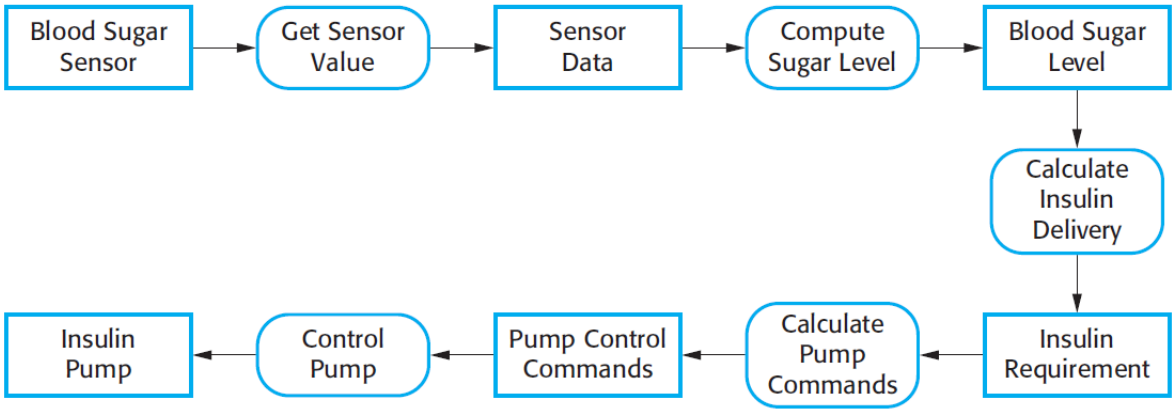
Behavioural models are models of the **dynamic** behaviour of the system as it is executing. They show **what happens or what is supposed to happen** when a system responds to a stimulus from its environment. We can think of these stimuli as being of two types

- Data Some data arrives that must be processed by the system.
- Events Some event happens that triggers system processing. Events may have associated data but this is not always the case.

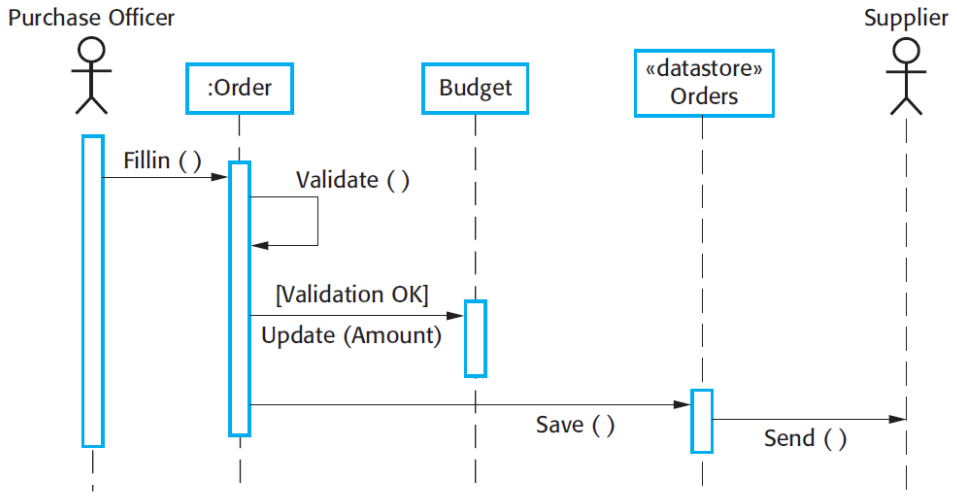
Data-driven modelling

- Data-driven models show **the sequence of actions involved in processing input data and generating an associated output**. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system.
- That is, they show the entire sequence of actions that take place from an input being processed to the corresponding output, which is the system's response.
- Data-driven models were amongst the first graphical software models Data-flow models are useful because tracking and documenting how the data associated with a particular process moves through the system helps analysts and designers understand what is going on.
- Data-flow diagrams are simple and intuitive and it is usually possible to explain them to potential system users who can then participate in validating the model.
- The UML does not support data-flow diagrams as they were originally proposed and used for modeling data processing. The reason for this is that DFDs focus on system functions and do not recognize system objects. However, because data-driven systems are so common in business, UML 2.0 introduced **activity diagrams**, which are similar to **data-flow diagrams**.
- Below figure shows an activity model of the insulin pump's operation

Interface College of Computer Applications (ICCA)



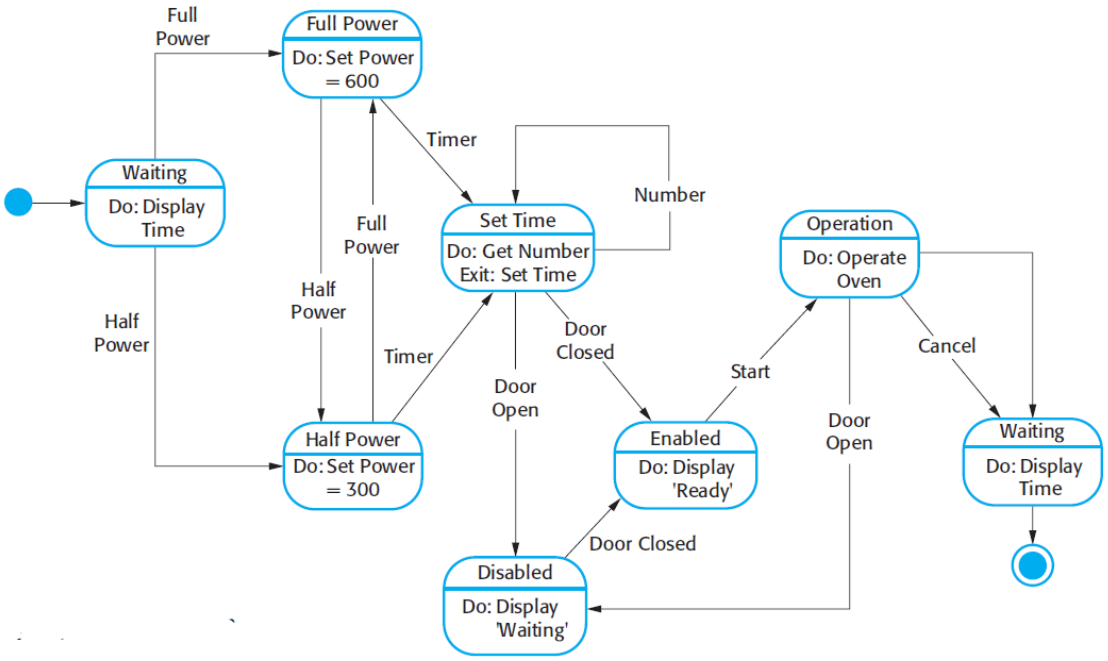
Sequence models highlight objects in a system, whereas data-flow diagrams highlight the functions.
Below figure illustrates this, using a sequence model of the processing of an order and sending it to a supplier



Interface College of Computer Applications (ICCA)

Event-driven modelling

- Event-driven modelling shows how a **system responds to external and internal events**. It assumes that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another.
- The UML supports event-based modelling using state diagrams, which were based on State charts. State diagrams show system states and events that cause transitions from one state to another. They do not show the flow of data within the system but may include additional information on the computations carried out in each state.
- Below State diagram of a microwave oven working with different states with below assumptions.
  - Select the power level (either half power or full power).
  - Input the cooking time using a numeric keypad.
  - Press Start and the food is cooked for the given time.



States and stimuli for the microwave oven is as below.

Interface College of Computer Applications (ICCA)

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.
Stimulus	**Description**
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

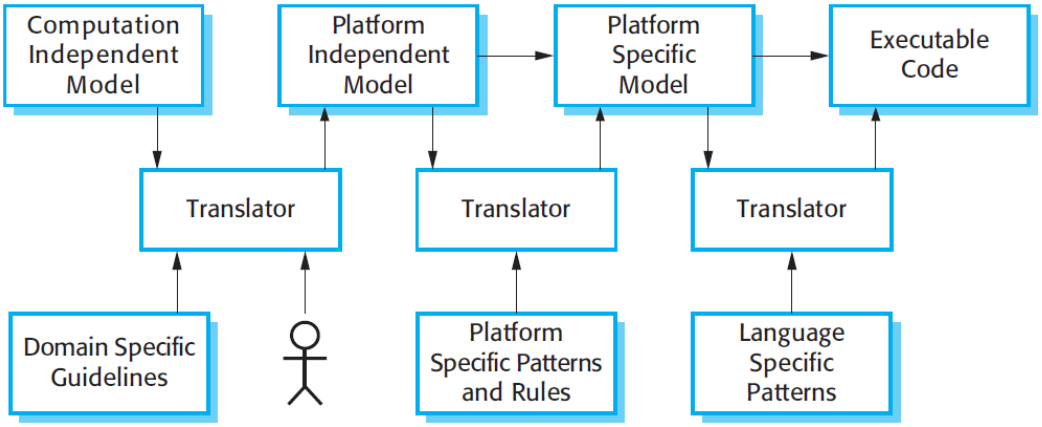
Interface College of Computer Applications (ICCA)

Model-driven engineering

- Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process.
- The programs that execute on a hardware/software platform are then generated automatically from the models.
- Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms.
- Model-driven engineering has its roots in model-driven architecture (MDA) ,MDA focuses on the design and implementation stages of software development whereas MDE is concerned with all aspects of the software engineering process. Therefore, topics such as model-based requirements engineering, software processes for model-based development, and model based testing are part of MDE but not, currently, part of MDA.

Model-driven architecture

- Model-driven architecture is a model-focused approach to software design and implementation that uses a sub-set of UML models to describe a system.
- Here, models at different levels of abstraction are created. From a high-level platform independent model it is possible, in principle, to generate a working program without manual intervention.



The MDA method recommends that three types of abstract system model should be produced.

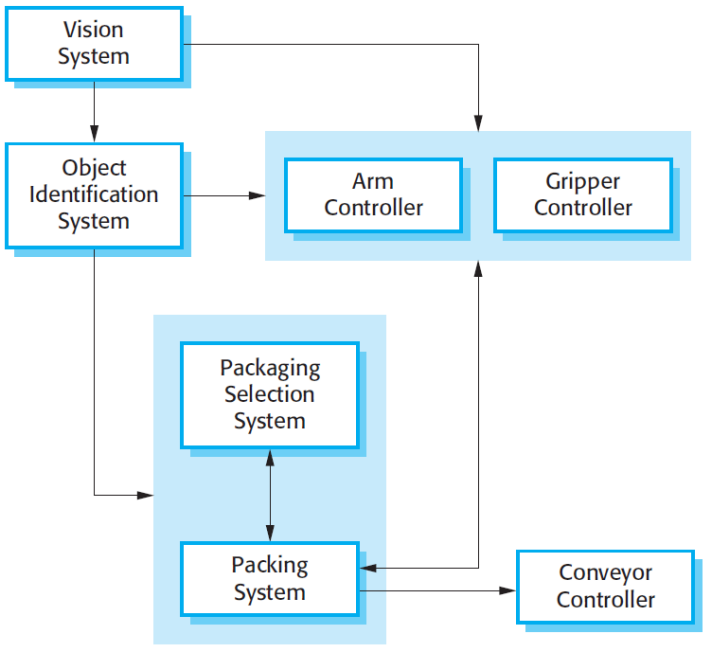
- A computation independent model (CIM) that models the important domain abstractions used in the system. CIMs are sometimes called domain models. You may develop several different CIMs, reflecting different views of the system. For example, there may be a security CIM in which you identify important security abstractions such as an asset and a role and a patient record CIM, in which you describe abstractions such as patients, consultations, etc.
- A platform independent model (PIM) that models the operation of the system without reference to its implementation. The PIM is usually described using UML models that show the static system structure and how it responds to external and internal events.
- Platform specific models (PSM) which are transformations of the platform independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer adding some platform specific detail. So, the first-level PSM could be middleware-specific but database independent. When a specific database has been chosen, a database specific PSM can then be generated.

Interface College of Computer Applications (ICCA)

UNIT- 4: ARCHITECTURAL DESIGN

Architectural design

- Architectural design is concerned with understanding **how a system should be organized and designing** the overall structure of that system.
- Architectural design is the **first stage** in the software design process.
- Architectural design is the critical link between **design and requirements engineering**, as it identifies the main structural components in a system and the relationships between them.
- The output of the architectural design process is an **architectural model that describes how the system is organized as a set of communicating components**.
- Below figure shows an abstract model of the architecture for a packing robot system that shows the components that must be developed. This robotic system can pack different kinds of object. It uses a vision component to pick out objects on a conveyor, identify the type of object and select the right kind of packaging. The system then moves objects from the delivery conveyor to be packaged. It places packaged objects on another conveyor. The architectural model shows these components and the links between them.



Inte cations (ICCA)

Three advantages of explicitly designing and documenting software architecture

- **Stakeholder communication** The architecture is a high-level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
- **System analysis** Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
- **Large-scale reuse** A model of a system architecture is a compact, manageable description of how a system is organized and how the components interoperate. The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse.

Architectural design decisions

- Software architectural design decisions are key guidelines to achieve **non-functional requirements of software systems** in the early stages of software development.
- These decisions are also important for justifying the modifications of dynamic architectures during software evolution in the operational phase.
- Incorporating reliability goals in software architectures is important for successful applications in **large and safety-critical** systems.
- During the architectural design process, system architects have to make a number of structural decisions that profoundly affect the system and its development process.
- Based on their knowledge and experience, they must consider the following fundamental questions about the system.

    - Is there a generic application architecture that can act as a template for the system that is being designed?
    - How will the system be distributed across several cores or processors?
    - What architectural patterns or styles might be used?
    - What will be the fundamental approach used to structure the system?
    - How will the structural components in the system be decomposed into subcomponents?
    - What strategy will be used to control the operation of the components in the system?
    - What architectural organization is best for delivering the non-functional requirements of the system?
    - How will the architectural design be evaluated?
    - How should the architecture of the system be documented?

- The architecture of a software system may be based on a particular architectural pattern or style. An architectural pattern is a description of a system organization such as a client–server organization or a layered architecture.
- Architectural patterns capture the essence of an architecture that has been used in different software systems.
- Architectural design focus on following nonfunctional requirements.
    - Performance
    - Security
    - Safety
    - Availability
    - Maintainability

Interface College of Computer Applications (ICCA)

Architectural views

Architecture views are representations of the overall architecture that are meaningful to one or more stakeholders in the system. The architect develops a set of views that will enable the architecture to be communicated to all the stakeholders.

- 4+1 is a view model used for "describing the architecture of software-intensive systems, based on the use of multiple, concurrent views".
- The views are used to describe the system from the viewpoint of different stakeholders, such as end-users, developers, system engineers, and project managers.
- The four views of the model are **logical, development, process and physical view**. In addition, selected use cases or scenarios are used to illustrate the architecture serving as the 'plus one' view. Hence, the model contains 4+1 views.

- Logical view: The logical view is concerned with the functionality that the system provides to end-users. UML diagrams are used to represent the logical view, and include class diagrams, and state diagrams.
- Process view: The process view deals with the dynamic aspects of the system, explains the system processes and how they communicate, and focuses on the run time behavior of the system. The process view addresses concurrency, distribution, integrator, performance, and scalability, etc. UML diagrams to represent process view include the sequence diagram, communication diagram, activity diagram.
- Development view: The development view (aka the implementation view) illustrates a system from a programmer's perspective and is concerned with software management. UML Diagrams used to represent the development view include the Package diagram and the Component diagram.
- Physical view: The physical view (aka the deployment view) depicts the system from a system engineer's point of view. It is concerned with the topology of software components on the physical layer as well as the physical connections between these components. UML diagrams used to represent the physical view include the deployment diagram.
- Scenarios: The description of an architecture is illustrated using a small set of use cases, or scenarios, which become a fifth view. The scenarios describe sequences of interactions between objects and between processes. They are used to identify architectural elements and to illustrate and validate the architecture design. They also serve as a starting point for tests of an architecture prototype. This view is also known as the use case view.

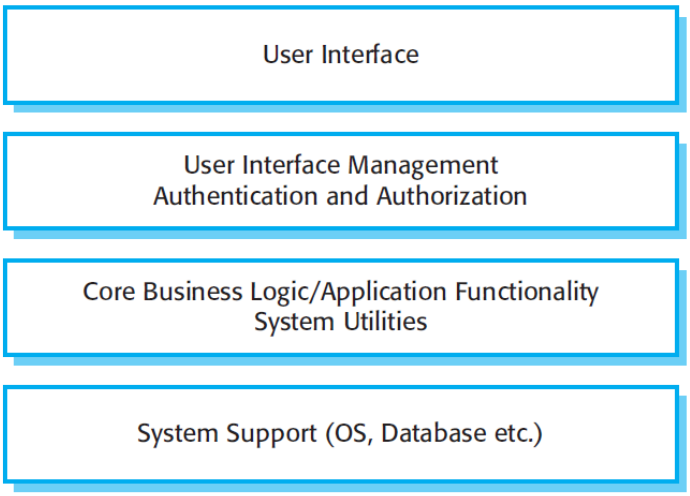
Architectural patterns

A software architecture pattern defines the high-level structure and organization of a software system. It outlines the fundamental components, their interactions, and the overall layout of the system. Architectural patterns guide decisions about the system's scalability, performance, and maintainability.

Layered architecture

- The notions of **separation and independence** are fundamental to architectural design because they allow changes to be localized.
- Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system.
- The layered architecture pattern is another way of achieving separation and independence as like **MVC** pattern.
- This pattern is shown in below figure.

ICCA Interface College of Computer Applications (ICCA)



- Here, the system functionality is **organized into separate layers**, and each layer only relies on the facilities and services offered by the layer **immediately beneath it**.
- This layered approach supports the **incremental development of systems**. As a layer is developed, some of the services provided by that layer may be made available to users.
- The architecture is also **changeable and portable**. So long as its interface is unchanged, a layer can be replaced by another, equivalent layer. Furthermore, when layer interfaces change or new facilities are added to a layer, only the adjacent layer is affected. As layered systems localize machine dependencies in inner layers, this makes it easier to provide multi-platform implementations of an application system.
- Only the **inner, machine-dependent layers** need to be re-implemented to take account of the facilities of a different operating system or database.
- Above figure is an example of a layered architecture with four layers. The lowest layer includes system support software—typically database and operating system support. The next layer is the application layer that includes the components concerned with the application functionality and utility components that are used by other application components. The third layer is concerned with user interface management and providing user authentication and authorization, with the top layer providing user interface facilities. Of course, the number of layers is arbitrary. Any of the layers in above figure could be split into two or more layers.
- **When used**: Used when building new facilities on **top of existing systems**; when the development is **spread across several teams** with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
- **Advantages**: Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
- **Disadvantages**: In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

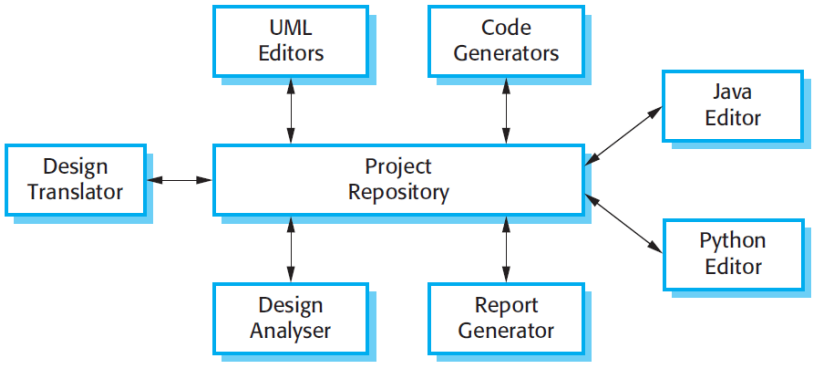
Interface College of Computer Applications (ICCA)

Repository architecture

- Repository pattern describes how a set of interacting components can **share data**.
- All data in a system is managed in a **central repository** that is accessible to all system components. Components do not interact directly, only through the repository.
- Most systems that use large amounts of data are organized around a

shared database or repository. This model is therefore suited to applications in which data is generated by one component and used by another. Examples of this type of system include command and control systems, management information systems, CAD systems, and interactive development environments for software.

- Below figure is an illustration of a situation in which a repository might be used.



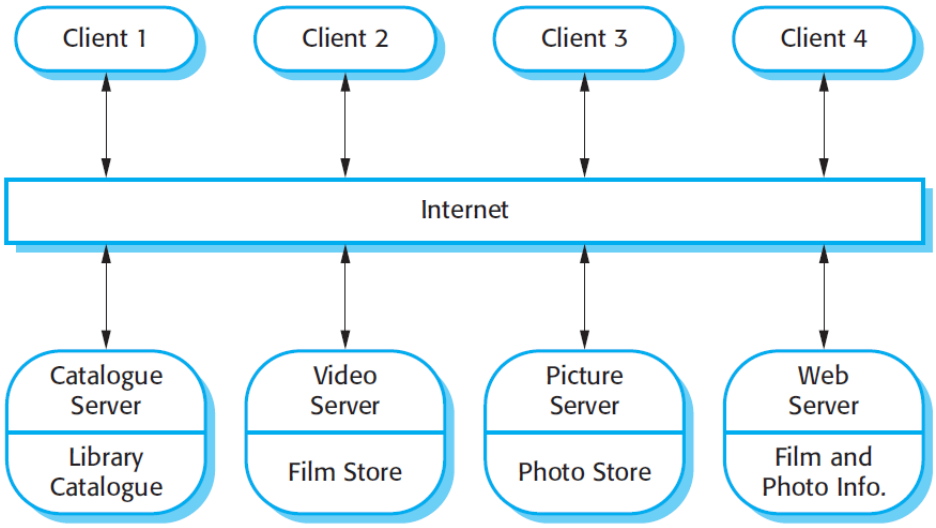
- This diagram shows an **IDE** that includes different tools to support model-driven development. The repository in this case might be a version-controlled environment that keeps track of changes to software and allows rollback to earlier versions.
- Repository pattern allows **implicit movement of data across the system as per the agreed repository data model**.
- Although it is possible to distribute a logically centralized repository, there may be problems with data **redundancy and inconsistency**.
- **When used**: You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
- **Advantages**: Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
- **Disadvantages**: The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Interface College of Computer Applications (ICCA)

Client–server architecture

- Client-server architecture is very commonly used in run-time organization for **distributed systems**.
- **In a client–server architecture, the functionality of the system is organized into services,**
  with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
- A system that follows the client–server pattern is organized as a set of services and associated **servers**, and clients that access and use the services. The major components of this model are as follows:
  - A set of **servers** that offer services to other components. Examples of servers include print servers that offer printing services, file servers that offer file management services, and a compile server, which offers programming language compilation services.

  - A set of **clients** that call on the services offered by servers. There will normally be several instances of a client program executing concurrently on different computers.
  - A network that allows the clients to access these services. Most client–server systems are implemented as distributed systems, connected using **Internet**. **protocols**.
- The Client–server pattern is as showed in below figure.

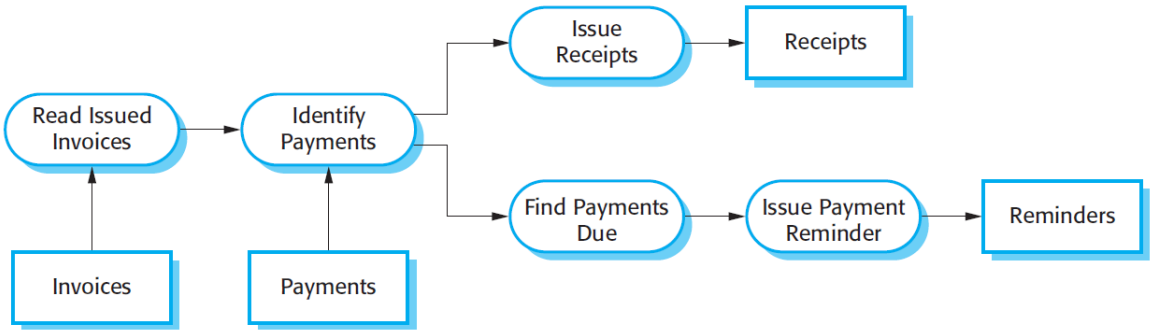


- Client-server architecture obey the **rules and conventions called protocols available in www, usually web protocol like http, https** plays dominant role while defining the rules.
- Addressing mechanism **called ip addressing** is used to uniquely identifying the nodes over the internet.
- The **catalog** must be able to deal with a variety of queries and provide links into the web information system that includes data about the film and video clips, and an e-commerce system that supports the sale of photographs, film, and video clips.
- **When used**: Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
- **Advantages**: Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
- **Disadvantages**: The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

Interface College of Computer Applications (ICCA)

Pipe and filter architecture

- **Pipe and Filter is another architectural pattern, which has independent entities called filters (components) which perform transformations on data and process the input they receive, and pipes, which serve as connectors for the stream of data being transformed, each connected to the next component in the pipeline.**
- The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
- An example of this type of system architecture, used in a batch processing application, is shown in below figure.



- An organization has issued invoices to customers. Once a week, payments that have been made are reconciled with the invoices. For those invoices that have been paid, a receipt is issued. For those invoices that have not been paid within the allowed payment time, a reminder is issued.

- **When used**: Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
- **Advantage**s: Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
- **Disadvantages**: The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparsed its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

Interface College of Computer Applications (ICCA)

DESIGN AND IMPLEMENTATION

Object-oriented design using the UML

Note:

The Unified Modelling Language (UML) is a graphical language for Object-oriented analysis and design (OOAD) that gives a standard way to write a software system's blueprint. It helps to visualize, specify, construct, and document the artifacts of an object-oriented system.

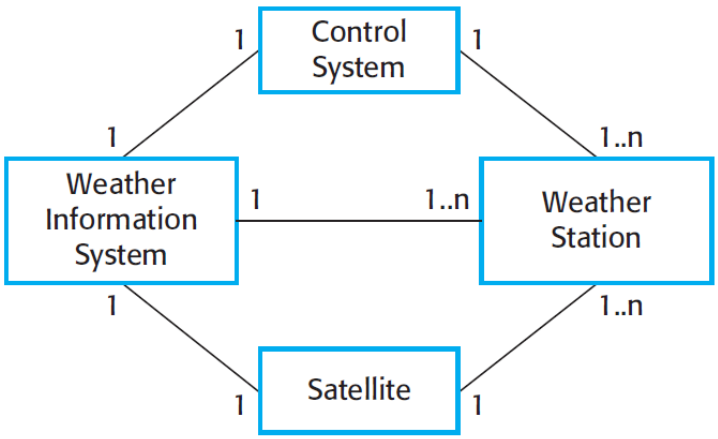
- An object-oriented system is made up of interacting objects that maintain their own local state and provide operations on that state.
- The representation of the state is private and cannot be accessed directly from outside the object.
- Object-oriented design processes involve designing object classes and the relationships between these classes.
- The Unified Modelling Language (UML) is inherently object-oriented modelling language and was designed for use in object-oriented software applications.
- The applications could be based on the object-oriented technologies recommended by the Object Management Group (OMG), which owns the UML.
- Object-oriented systems are easier to change than systems developed using functional approaches.
- Objects include both data and operations to manipulate that data.
- They may therefore be understood and modified as stand-alone entities.
- Changing the implementation of an object or adding services should not affect other system objects.
- To develop a system design from concept to detailed, object-oriented design, there are several things that you need to do
  - Understand and define the context and the external interactions with the system.
  - Design the system architecture.
  - Identify the principal objects in the system.
  - Develop design models.
  - Specify interfaces.

System context and interactions

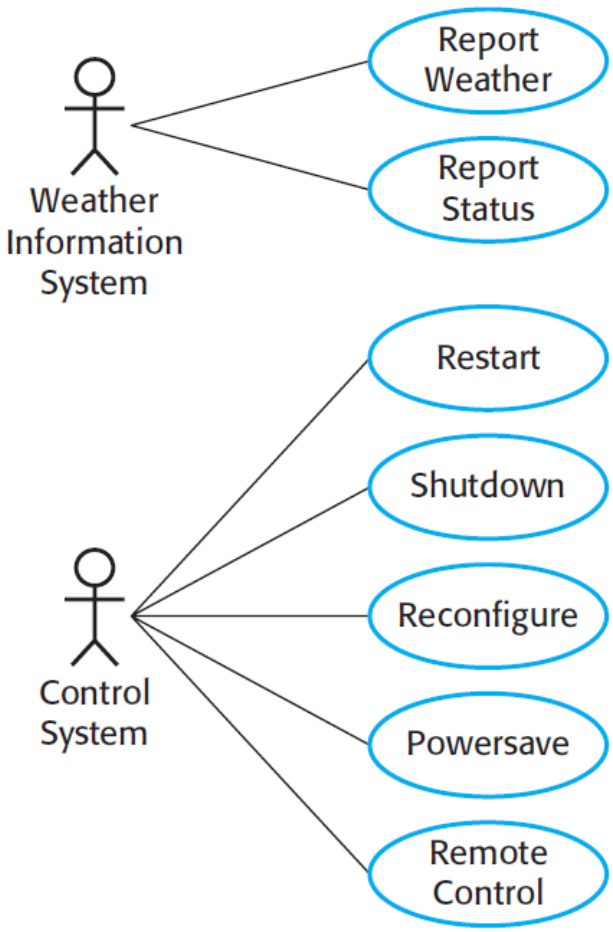
Interface College of Computer Applications (ICCA)

- **System context models and interaction models present complementary views of the relationships between a system and its environment.**
  - A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
  - An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.

- The context model of a system may be represented using associations. Associations simply show that there are some relationships between the entities involved in the association and associations are as shown in below figure 1 and 2, which shows that the systems in the environment of each weather station are a weather information system, an onboard satellite system, and a control system.
- Figure_1 shows System context for the weather station.



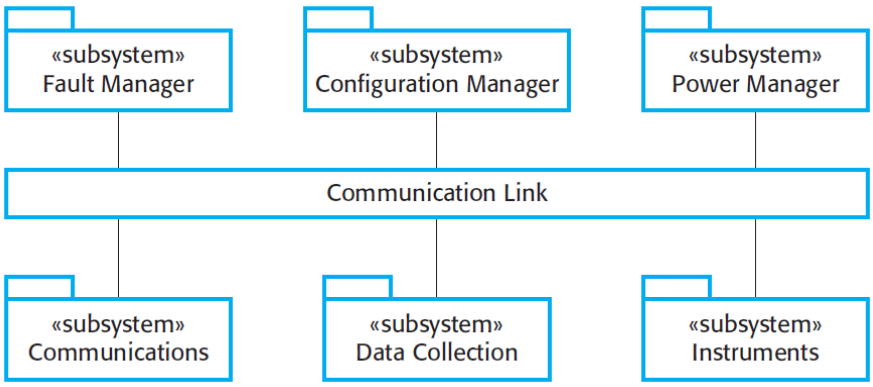
- Figure_2 shows Weather Control station use cases.



Interfa tions (ICCA)

Architectural design

- **Once the interactions between the software system and the system's environment have been defined, we use this information as a basis for designing the system.**
- Architecture along with the use of general knowledge of the principles of architectural design and with more detailed domain knowledge.
- Next phase identify the major components that make up the system and their interactions, and then may organize the components using an architectural pattern such as a layered or client–server model.
- The high-level architectural design for the weather station software is shown in below figure. The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure, shown as the Communication link in below figure. Each subsystem listens for messages on that infrastructure and picks up the messages that are intended for them.



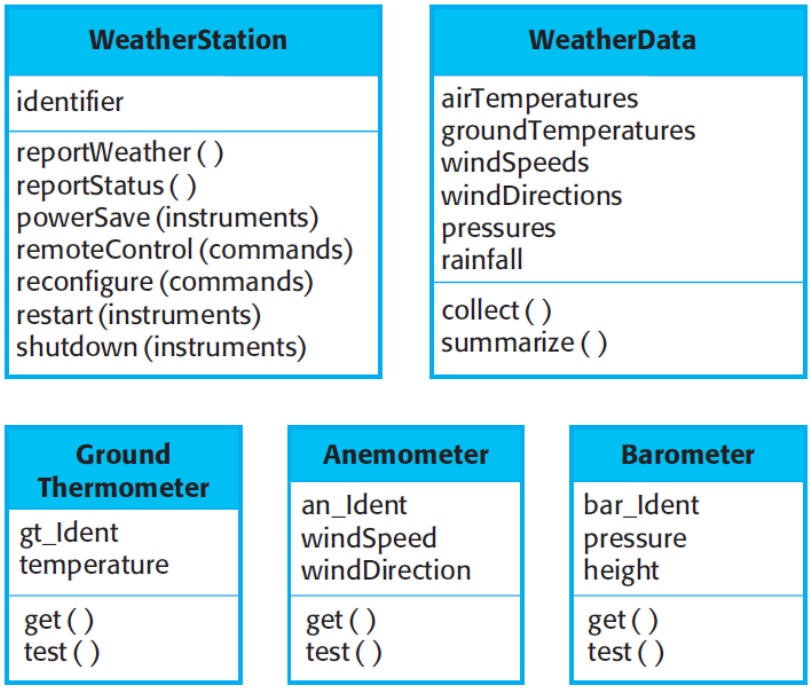
Object class identification

- **Object class identification is the act of identifying the attributes of entities and discovery of the functionalities associated with the entity and defining the relationship between different entities of the system and represent them in the form of classes and creating the objects of the respective classes.**
- There have been various proposals made about how to identify object classes in object-oriented systems

    - Use a grammatical analysis of a natural language description of the system to be constructed. Objects and attributes are nouns; operations or services are verbs.
    - Use tangible entities (things) in the application domain such as aircraft, roles such as manager or doctor, events such as requests, interactions such as meetings, locations such as offices, organizational units such as companies, and so on.
    - Use a scenario-based analysis where various scenarios of system use are identified and analysed in turn. As each scenario is analysed, the team responsible for the analysis must identify the required objects, attributes, and operations.

Interface College of Computer Applications (ICCA)

- In the wilderness weather station, object identification is based on the tangible hardware in the system is shown in below figure.



Design models

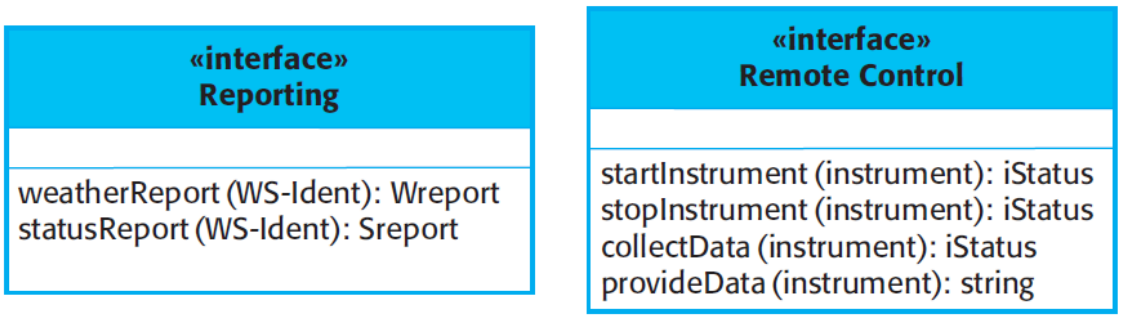
- **Design or system models show the objects or object classes in a system. They also show the associations and relationships between these entities.**
- These models are the bridge between the system requirements and the implementation of a system. They have to be abstract so that unnecessary detail doesn't hide the relationships between them and the system requirements. However, they also have to include enough detail for programmers to make implementation decisions.
- When you use the UML to develop a design, you will normally develop two kinds of design model.
  - **Structural models**, which describe the static structure of the system using object classes and their relationships. Important relationships that may be documented at this stage are generalization (inheritance) relationships, uses/used-by relationships, and composition relationships.
  - **Dynamic models**, which describe the dynamic structure of the system and show the interactions between the system objects. Interactions that may be documented include the sequence of service requests made by objects and the state changes that are triggered by these object interactions.
- In the early stages of the design process, I think there are three models that are particularly useful for adding detail to use case and architectural models:
  - **Subsystem models**, which that show logical groupings of objects into coherent subsystems. These are represented using a form of class diagram with each subsystem shown as a package with enclosed objects. Subsystem models are static (structural) models.
  - **Sequence models**, which show the sequence of object interactions. These are represented using a UML sequence or a collaboration diagram. Sequence models are dynamic models.
  - **State machine model**, which show how individual objects change their state in

Interface College of Computer Applications (ICCA)

response to events. These are represented in the UML using state diagrams. State machine models are dynamic models.

Interface specification

- An important part of any design process is the specification of the **interfaces between the components in the design**.
- We need to specify interfaces so that objects and subsystems can be designed in parallel. Once an interface has been specified, the developers of other objects may assume that interface will be implemented.
- Interface design is concerned with specifying the detail of the interface to an object or to a group of objects. **This means defining the signatures and semantics of the services that are provided by the object or by a group of objects**.
- Interfaces can be specified in the **UML using the same notation as a class diagram**. However, there is **no attribute section and the UML stereotype ‹‹interface››** should be included in the name part. The semantics of the interface may be defined using the **object constraint language (OCL).**
- Below figure shows two interfaces that may be defined for the weather station. The left-hand interface is a reporting interface that defines the operation names that are used to generate weather and status reports.



- We should not include details of the data representation in an interface design, as attributes are not defined in an interface specification.
- However, we should include operations to access and update data. As the data representation is hidden, it can be easily changed without affecting the objects that use that data.

Interface College of Computer Applications (ICCA)

Design patterns

- **Patterns and Pattern Languages are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience.**
- To use patterns in our design, we need to recognize that any design problem we are facing may have an associated pattern that can be applied. Examples of such problems, documented in the '**Gang of Four's original patterns book**, include
  - Tell several objects that the state of some other object has changed (Observer pattern).
  - Tidy up the interfaces to several related objects that have often been developed incrementally (Façade pattern).
  - Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (Iterator pattern).

- Allow for the possibility of extending the functionality of an existing class atrun-time (Decorator pattern).

Note: Software architecture focuses more on the abstract view of idea while Design pattern focuses on the implementation view of idea.

Implementation issues

Some of the pointes we need to consider during implementation is as follows.

- **Reuse** Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
- **Configuration management** During the development process, many different versions of each software component are created. If you don't keep track of these versions in a configuration management system, you are liable to include the wrong versions of these components in your system.
- **Host-target development** Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system). The host and target systems are sometimes of the same type but, often they are completely different.



Interface College of Computer Applications (ICCA)

Unit - V: SOFTWARE TESTING

Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use. When you test software, you execute a program using artificial data. You check the results of the test run for errors, anomalies, or information about the program's non-functional attributes.

Testing can only show the presence of errors, not their absence.

The testing process has two distinct goals as below.

- **To demonstrate to the developer and the customer that the software meets its requirements.** For custom software, this means that there should be at least one test for every requirement in the requirements document. For generic software products, it means that there should be tests for all of the system features, plus combinations of these features, that will be incorporated in the product release.
- **To discover situations in which the behaviour of the software is incorrect, undesirable, or does not conform to its specification.** These are a consequence of software defects. Defect testing is concerned with rooting out undesirable system behaviour such as system crashes, unwanted interactions with other systems, incorrect computations, and data corruption.

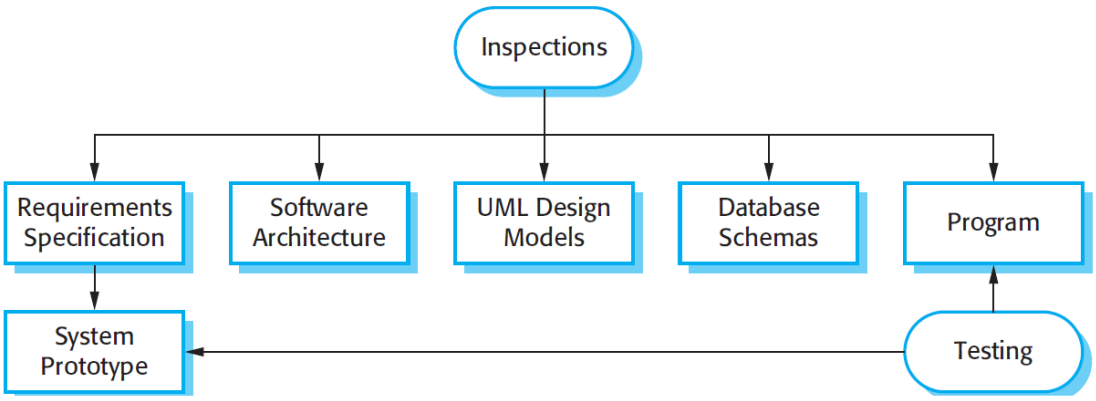
The first goal leads to validation testing, where you expect the system to perform correctly using a given set of test cases that reflect the system's expected use. The second goal leads to defect testing.
The ultimate goal of verification and validation processes is to establish confidence that the software system is 'fit for purpose'.

Note:

'Validation: Are we building the right product?'
'Verification: Are we building the product right?'

As well as software testing, the verification and validation process may involve software inspections and reviews. **Inspections and reviews** analyse and check the system requirements, design models, the program source code, and even proposed system tests. These are so-called 'static' V & V techniques in which you don't need to execute the software to verify it. Below figure shows that software inspections and testing support V & V at different stages in the software process. The arrows indicate the stages in the process where the techniques may be used.

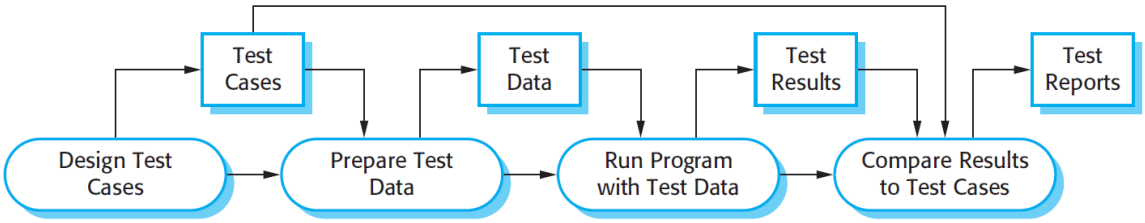
Interface College of Computer Applications (ICCA)



Test planning

- Test planning is concerned with **scheduling** and **resourcing** all the activities in the testing process.
- It involves defining the testing process, considering the people and the time available. Usually, a test plan will be created, which defines what is to be tested, the predicted testing schedule, and how tests will be recorded.
- For critical systems, the test plan may also include details of the tests to be run on the software.

Below figure is an abstract model of the 'traditional' testing process, as used in plan driven development.



Typically, a commercial software system has to go through three stages of testing

- **Development testing**, where the system is tested during development to discover bugs and defects. System designers and programmers are likely to be involved in the testing process.
- **Release testing**, where a separate testing team tests a complete version of the system before it is released to users. The aim of release testing is to check that the system meets the requirements of system stakeholders.
- **User testing**, where users or potential users of a system test the system in their own environment. For software products, the 'user' may be an internal marketing group who decides if the software can be marketed, released, and sold.
  Acceptance testing is one type of user testing where the customer formally tests a system to decide if it should be accepted from the system supplier or if further development is required.

Interface College of Computer Applications (ICCA)

Development testing

Development Testing refers to the testing activities performed during the development phase of the software lifecycle. It is primarily carried out by developers to ensure the individual components or units of the software function correctly before integrating them into the larger system.

- Development testing includes all testing activities that are **carried out by the team developing the system.**
- The tester of the software is usually the programmer who developed that software, although this is not always the case. Some development processes use **programmer/tester pairs** where each programmer has an associated tester who develops tests and assists with the testing process.
- For critical systems, a more formal process may be used, with a **separate testing group** within the development team. They are responsible for developing tests and maintaining detailed records of test results.
- During development, testing may be carried out at three levels of granularity
    - **Unit testing**, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
    - **Component testing**, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
    - **System testing**, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

Note:

Development testing is primarily a defect testing process, where the aim of testing is to discover bugs in the software. It is therefore usually interleaved with debugging—the process of locating problems with the code and changing the program to fix these problems.

Debugging is the process of fixing errors and problems that have been discovered by testing. Using information from the program tests, debuggers use their knowledge of the programming language and the intended outcome of the test to locate and repair the program error. This process is often supported by interactive debugging tools that provide extra information about program execution.

Unit Testing

Interface College of Computer Applications (ICCA)

- This type of testing is performed by developers before the setup is handed over to the testing team to formally execute the test cases. **Unit testing is performed by the respective developers on the individual units of source code assigned areas**. The developers use test data that is different from the test data of the quality assurance team.
- The goal of unit testing is to isolate each part of the program and show that individual parts are correct in terms of requirements and functionality.

Limitations of Unit Testing

- Testing cannot catch each bug in an application. It is impossible to evaluate every execution path in every software application. The same is the case with unit testing.
- There is a limit to the number of scenarios and test data that a developer can use to verify a source code. After having exhausted all the options, there is no choice but to stop unit testing and merge the code segment with other units.

Advantages of Unit Testing
Early Defect Detection:
Fixing issues at the unit level is less expensive than addressing them in later phases.
Improved Code Quality:
Encourages developers to write cleaner and more modular code

Disadvantages:
Cannot Catch Integration Issues:
Unit testing focuses only on individual units, not interactions between units.
Time-Consuming for Complex Applications:
Writing comprehensive unit tests can require significant effort.

Choosing unit test cases

5. Error Guessing
Principle: Use the tester's experience to guess where errors are most likely to occur and focus on testing those areas.
How It Works: Test cases are chosen based on the tester's intuition and experience, focusing on known problem areas, such as:

Testing is expensive and time consuming, so it is important that you choose effective unit test cases. Effectiveness, in this case, means two things.

- The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do.
- If there are defects in the component, these should be revealed by test cases.

Two possible strategies here that can be effective in helping you choose test cases. These are:

1. Boundary Value Analysis (BVA)
Principle: Focus on testing the boundary values, where the program is most likely to fail.
How It Works: Test the values at the boundaries of the input domain as well as just inside and outside of the boundaries.
Examples:
If a function accepts values between 1 and 100:
Test: 1, 100 (boundary values)
Test: 0, 101 (just outside the boundaries)
Test: 2, 99 (just inside the boundaries)

- **Partition testing**, where you identify groups of inputs that have common characteristics and should be processed in the same way. You should choose tests from within each of these groups.
- **Guideline-based testing**, where you use testing guidelines to choose test cases. These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.
-

2. Equivalence Partitioning
Principle: Divide input data into equivalent classes where the system should behave similarly. Select one representative value from each class to reduce the number of test cases.
How It Works: Instead of testing every single value, test a few that represent different categories or partitions of the input space.

Path testing

Path testing is a structural testing technique used in software testing to ensure that every possible path through the code is executed at least once. It is primarily focused on ensuring that all logical branches, loops, and decision points in the code are covered during testing.

Path testing is a testing strategy that aims to exercise every independent execution path through a component or program. If every independent path is executed, then all statements in the component must have been executed at least once. All conditional statements are tested for both true and false cases. In an object-oriented development process, path testing may be used when testing the methods associated with objects.

Component testing

- **Component testing, also known as program or module testing, is done after unit testing. In this type of testing those test objects can be tested independently as a component without integrating with other components e.g. modules, classes, objects, and programs. This testing is done by the development team.**
- Component testing is like unit testing with the difference that the developer uses real data instead of dummy data for testing of the written code.
- Suppose there is a software application which consists of five components modules. The testing of each module is done independently by the developer as part of the development cycle before it is ready for integration testing. By doing component testing, bugs can be found at a very early stage in the cycle and helps save time.
- Debugging tools or unit test structure tools are used for this type of testing since this is done by programmers on the code written by them and with the support of integrated development environment. Defects are fixed as soon as possible when they are found without formally recording incidents.
- Component testing plays an important role in finding the issue. Before we proceed with the integration testing it's always advised to do the component testing in order to ensure that each module of an application is working correctly and as per requirement.

Interface College of Computer Applications (ICCA)

Types of Integration Testing:
Top-Down Integration Testing:
Testing begins from the top-level modules (the main components of the system) and moves downward toward the lower-level components.

Bottom-Up Integration Testing:
Testing starts from the lowest-level modules (the ones closer to the core logic) and proceeds upwards.

Sandwich (Hybrid) Integration Testing:
A combination of both top-down and bottom-up approaches.

Incremental Integration Testing:
Involves adding one module at a time and testing the interactions between modules.
This can be done either top-down or bottom-up.

Big Bang Integration Testing:
All components or modules are integrated at once, and then the system is tested as a whole.
This approach is less common due to the difficulties in identifying which specific module causes any issues.

Integration Testing

Integration Testing is a level of software testing where individual units or components of a software application are combined and tested as a group. The main objective of integration testing is to verify the interactions between different modules or components to ensure they work together as expected.
Tests interactions between different components or modules

- **Integration testing is defined as the testing of combined parts of an application to determine if they function correctly.**
- Integration testing can be done in two ways: Bottom-up integration testing and Top-down integration testing.
- **Bottom-up integration**
  This testing begins with unit testing, followed by tests of progressively higher-level combinations of units called modules or builds.
- **Top-down integration**
  In this testing, the highest-level modules are tested first and progressively, lower-level modules are tested thereafter.
  In a comprehensive software development environment, bottom-up testing is usually done first, followed by top-down testing. The process concludes with multiple tests of the complete application, preferably in scenarios designed to mimic actual situations.

System Testing

It is performed after integration testing and involves testing the complete, integrated system in an environment that closely resembles the production environment.
The main objective of system testing is to verify that the software works as expected in its entirety and that it behaves as required under different conditions

System testing tests the system as a whole. Once all the components are integrated, the application as a whole is tested rigorously to see that it meets the specified Quality Standards. This type of testing is performed by a specialized testing team.

System testing is important because of the following reasons.

- System testing is the first step in the Software Development Life Cycle, where the application is tested.
- The application is tested thoroughly to verify that it meets the functional and technical specifications.
- The application is tested in an environment that is very close to the production environment where the application will be deployed.
- System testing enables us to test, verify, and validate both the business requirements as well as the application architecture.

Test-driven development

- **Test-driven development (TDD) is a software development process relying on software requirements being converted to test cases before software is fully developed and tracking all software development by repeatedly testing the software against all test cases. This is as opposed to software being developed first and test cases created later.**
- Test-driven development is related to the test-first programming concepts of extreme programming.
- Programmers also apply the concept to improving and debugging legacy code developed with older techniques.
- The fundamental TDD process is shown in below figure and the steps in the process are as follows.

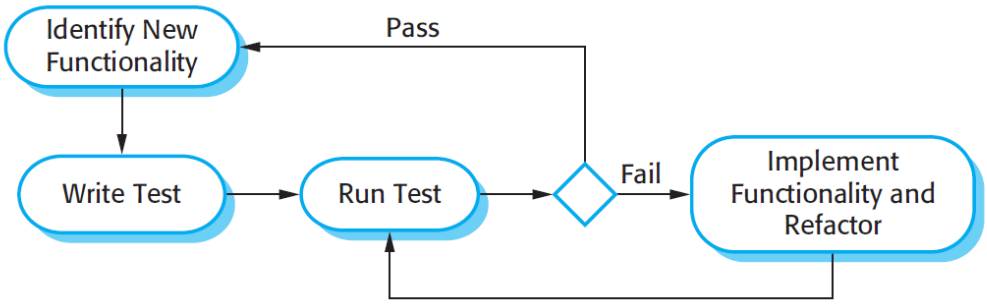
Test-Driven Development (TDD) is a software development methodology where tests are written before writing the actual functional code.
Write Tests First:
In TDD, you write the test cases before implementing the functionality. These tests define the expected behavior of the software.

Failing Test:
Initially, the test will fail because no implementation exists. This step ensures that the test is valid and captures the intended behavior.

Code to Pass Tests:
Write the minimal amount of code required to pass the failing test. Focus only on the functionality specified by the test.

Refactor:
After the test passes, improve the code's structure, readability, and performance while ensuring the test still passes. This step ensures clean and maintainable code.

Repeat:
Add new tests for additional functionality and continue the cycle.



- We start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- We write a test for this functionality and implement this as an automated test.
- This means that the test can be executed and will report whether or not it has passed or failed.
  We then run the test, along with all other tests that have been implemented.
- Initially, you have not implemented the functionality so the new test will fail.
- This is deliberate as it shows that the test adds something to the test set.
  We then implement the functionality and re-run the test. This may involve refactoring existing code to improve it and add new code to what's already there.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.

Release testing

Release testing is a type of software testing conducted to ensure that the software system is ready for deployment or release to end users. Release testing is typically performed in an environment that closely resembles the production environment to simulate real-world usage.

- **Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.**
- Normally, the system release is for customers and users. In a complex project, however, the release could be for other teams that are developing related systems. For software products, the release could be for product management who then prepare it for sale.
- There are two important distinctions between release testing and system testing during the development process.
  - A separate team that has not been involved in the system development should be responsible for release testing.
  - System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).
- The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use. If so, it can be released as a product or delivered to the customer.
- Release testing, therefore, has to show that the system delivers its specified functionality, performance, and dependability, and that it does not fail during normal use. It should take into account all of the system requirements, not just the requirements of the end-users of the system.
- Release testing is usually a black-box testing process where tests are derived from the system specification. The system is treated as a black box whose behaviour can only be determined by studying its inputs and the related outputs. Another name for this is 'functional testing', so-called because the tester is only concerned with functionality and not the implementation of the software.

Interface College of Computer Applications (ICCA)

User testing

What is User Testing?
User testing (also called usability testing) is a process where real users interact with a software application or product to evaluate its usability, functionality, and overall user experience. The goal is to identify areas where the product can be improved to better meet the needs of its target audience

User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing. This may involve formally testing a system that has been commissioned from an external supplier or could be an informal process where users experiment with a new software product to see if they like it and that it does what they need.

In practice, there are three different types of users testing.

- **Alpha testing**, where users of the software work with the development team to test the software at the developer's site.
- **Beta testing**, where a release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**, where customers test a system to decide whether it is ready to be accepted from the system developers and deployed in the customer environment.

Alpha testing :
is an internal testing phase performed by the development team and/or quality assurance (QA) team before the software is released to external users. It is conducted in a controlled environment to identify and fix bugs.

Key Features:
Performed By: Internal staff (developers, testers).
Environment: Controlled (similar to the production environment).

Beta testing is a real-world testing phase performed by a select group of external users (beta testers) before the official product launch. It provides feedback from actual users in a live environment.

Key Features:
Performed By: External users (target audience).
Environment: Real-world or live environment.

Acceptance testing is the final testing phase conducted to determine whether the software meets the specified business requirements and is ready for deployment. It is often performed by the client or end-users.

Key Features:
Performed By: End-users or client representatives.
Environment: Simulated production or actual production environment.

Black Box Testing is a software testing method that examines the functionality of an application without knowing its internal workings, structure, or code. Testers focus on providing inputs and checking the corresponding outputs against expected results.
White Box Testing (also called Clear Box or Glass Box Testing) is a testing approach where the tester has access to the software's internal code, logic, and structure. It involves verifying the flow of inputs through the code and ensuring all paths and conditions are executed.

Interface College of Computer Applications (ICCA)